

hãy giải thích tất cả khái niệm từ chi tiết đến nâng cao về các vấn đề sau:

1. redis: mục đích, các sử dụng, áp dụng trong hệ thống phân tán, redis cluster, quản lý redis, ...
2. xử lý file trong ứng dụng java, từ file nhỏ đến file lớn, best practice trong doanh nghiệp
3. multithread các vấn đề thường gặp, blocking queue, giải thích cơ chế mới của thread trên jdk21 so với cơ chế cũ trên jdk về trước,
4. cơ chế batch job trong java, kết hợp với jenkins để làm gì, mục đích hay dùng của batch job này, các vấn đề gặp

Xem thêm ▾

Mình sẽ đi theo hướng "senior Java/banking thực chiến": giải thích gốc rễ vấn đề, sau đó mới lên thiết kế production, lỗi thường gặp và best practice. Với JDK 21, Kubernetes, Jenkins, Redis Cluster là các phần có thể thay đổi theo version, mình sẽ kiểm tra tài liệu chính thức để tránh nói sai version/cơ chế.

Đã suy nghĩ trong 18s >

Dưới đây là bản giải thích theo hướng **senior Java / banking / hệ thống lớn**. Mình sẽ không chỉ định nghĩa thuật ngữ, mà sẽ đi từ **vấn đề gốc rễ** → **cách thiết kế** → **áp dụng thực tế** → **lỗi production hay gặp** → **best practice**.

Tổng quan trước: 7 chủ đề này liên quan với nhau như thế nào?

Trong một hệ thống lớn, ví dụ banking, payment, e-commerce, insurance, trading, hoặc logistics, các thành phần này thường không đứng riêng lẻ.

Một request đi vào hệ thống có thể đi qua:

Client / Mobile App / Web

|

v

API Gateway

|

v



```

Java Microservices
|
+---> Redis cache / distributed lock / rate limit / session
|
+---> Database
|
+---> Kafka / Queue
|
+---> Batch Job xử lý cuối ngày
|
v
File storage / Report / Reconciliation

```

Còn việc build, test, deploy hệ thống sẽ đi qua:

```

Developer push code
|
v
Git
|
v
Jenkins CI/CD
|
v
Build Docker image
|
v
Push image registry
|
v
Deploy lên Kubernetes
|
v
Scale, monitor, rollback

```



Nói ngắn gọn:

Redis giúp hệ thống nhanh hơn và xử lý phân tán tốt hơn.

File processing giúp xử lý dữ liệu lớn, report, import/export.

Multithread giúp Java xử lý nhiều việc đồng thời.

Batch job giúp xử lý dữ liệu theo lô, thường dùng cho cuối ngày hoặc xử lý nền.

Jenkins + CI/CD giúp tự động build, test, deploy.

API Gateway là cổng vào của hệ thống microservice.

Kubernetes giúp scale, deploy, self-healing và vận hành production.

1. Redis

1.1 Redis là gì?

Redis là một **in-memory data store**, nghĩa là dữ liệu chủ yếu được lưu trong RAM thay vì lưu trực tiếp trên disk như database truyền thống.

Vì RAM nhanh hơn disk rất nhiều, Redis thường được dùng cho những dữ liệu cần truy cập cực nhanh như:

```
cache
session
token
rate limit
distributed lock
leaderboard
queue nhẹ
idempotency key
temporary state
real-time counter
```



Redis không chỉ là key-value đơn giản. Nó hỗ trợ nhiều cấu trúc dữ liệu như:

```
String
Hash
List
Set
Sorted Set
Stream
Bitmap
HyperLogLog
Geo
```



Trong hệ thống lớn, Redis thường không thay thế database chính. Nó nằm ở giữa application và database để giảm tải, tăng tốc, hoặc điều phối trạng thái phân tán.

Redis Cluster hỗ trợ tự động chia dữ liệu trên nhiều node bằng cơ chế sharding; Redis dùng 16,384 hash slots và phân phối các slot này qua các primary shard.  Redis +1

1.2 Mục đích chính của Redis

Mục đích 1: Cache dữ liệu đọc nhiều

Ví dụ hệ thống banking có API lấy thông tin ngân hàng, tỷ giá, cấu hình phí giao dịch, danh sách ngân hàng thụ hưởng.

Nếu mỗi request đều đọc database:

```
Client request
  -> Service
    -> Database
```



Khi traffic tăng cao, database bị nghẽn.

Dùng Redis:

```
Client request
  -> Service
    -> Redis
      -> Nếu có dữ liệu: trả luôn
      -> Nếu không có: đọc DB, ghi Redis, rồi trả response
```



Đây gọi là **cache-aside pattern**.

Pseudo flow:

```
UserProfile profile = redis.get("user:" + userId);

if (profile == null) {
    profile = database.findUser(userId);
    redis.set("user:" + userId, profile, Duration.ofMinutes(10));
}

return profile;
```

Best practice trong dự án lớn:

Không cache mọi thứ. Chỉ cache dữ liệu:

```
đọc nhiều
ít thay đổi
tính toán tốn kém
database query nặng
không quá nhạy cảm hoặc đã được bảo vệ tốt
```



Ví dụ nên cache:

```
bank configuration
currency metadata
product catalog
user permission short-lived
OTP attempt counter
rate limit counter
```



Ví dụ phải cẩn thận khi cache:

số dư tài khoản
trạng thái giao dịch
thông tin định danh nhạy cảm
dữ liệu có yêu cầu consistency cao



1.3 Cache có những vấn đề gì?

Redis nhanh, nhưng cache không đơn giản. Production hay chết vì cache sai hơn là vì không có cache.

Vấn đề 1: Cache stale

Database đã thay đổi nhưng Redis vẫn giữ dữ liệu cũ.

Ví dụ:

User đổi hạn mức chuyển tiền từ 10 triệu lên 100 triệu.
DB đã update.
Redis vẫn giữ limit cũ 10 triệu.
API trả sai.



Cách xử lý:

Set TTL ngắn
Delete cache khi update DB
Dùng event để invalidate cache
Versioning cache key



Ví dụ:

```
@Transactional
public void updateLimit(String userId, BigDecimal newLimit) {
    database.updateLimit(userId, newLimit);
    redis.delete("user-limit:" + userId);
}
```

Nhưng có vấn đề: nếu DB update thành công, Redis delete thất bại thì sao?

Trong hệ thống lớn, thường dùng thêm event:

Update DB
-> publish event UserLimitChanged



-> cache invalidation consumer xóa cache

Vấn đề 2: Cache penetration

Client request key không tồn tại liên tục, ví dụ hacker request userId random. Redis không có, request đập xuống DB liên tục.

Cách xử lý:

Cache null value trong thời gian ngắn
Bloom Filter
Validate input trước khi query DB
Rate limit



Ví dụ:

```
User user = redis.get("user:" + id);

if (user == null) {
    user = db.findById(id);

    if (user == null) {
        redis.set("user:" + id, NULL_MARKER, Duration.ofMinutes(1));
        return null;
    }

    redis.set("user:" + id, user, Duration.ofMinutes(10));
}
```

Vấn đề 3: Cache breakdown / hot key

Một key rất hot, ví dụ currency-rate:USD , hết TTL đúng lúc traffic cao. Hàng ngàn request cùng miss Redis và cùng đập DB.

Cách xử lý:

Mutex lock khi rebuild cache
Refresh ahead
TTL random
Local cache tầng 1



Ví dụ TTL random:

```
int ttl = 300 + random.nextInt(60); // 5 phút + random 0-60s
redis.set(key, value, Duration.ofSeconds(ttl));
```

Mục đích là tránh hàng loạt key expire cùng lúc.

Vấn đề 4: Cache avalanche

Nhiều key expire cùng thời điểm hoặc Redis down, toàn bộ traffic đổ xuống DB.

Cách xử lý:

```
TTL random
Circuit breaker
Rate limit
Fallback
Redis HA / Cluster
Local cache tạm thời
DB protection
```



Trong banking, khi Redis down, không được để DB chết theo. Hệ thống phải có cơ chế giảm tải.

Ví dụ:

```
Redis fail
-> không retry vô hạn
-> dùng timeout ngắn
-> fallback dữ liệu mặc định nếu an toàn
-> reject request ít quan trọng
-> bảo vệ request quan trọng
```



1.4 Redis trong hệ thống phân tán

Redis rất hay dùng để giải quyết các bài toán mà một service instance đơn lẻ không làm được.

Giả sử service chạy 20 pod trên Kubernetes:

```
payment-service pod 1
payment-service pod 2
payment-service pod 3
```



...

payment-service pod 20

Mỗi pod có memory riêng. Nếu dùng ConcurrentHashMap local thì pod 1 biết, pod 2 không biết.

Redis là nơi chia sẻ trạng thái giữa nhiều pod.

1.5 Redis dùng làm distributed lock

Ví dụ một giao dịch transactionId = TX123 được gửi vào hệ thống 2 lần do retry.

Nếu 2 pod cùng xử lý:

Pod A xử lý TX123
Pod B cũng xử lý TX123



Có thể bị double charge.

Dùng Redis lock:

SET lock:TX123 randomValue NX PX 30000



Ý nghĩa:

NX: chỉ set nếu key chưa tồn tại
PX 30000: lock tự hết hạn sau 30 giây
randomValue: để chỉ owner thật sự mới được unlock



Flow:

```
boolean locked = redis.setIfAbsent(  
    "lock:transaction:" + txId,  
    randomToken,  
    Duration.ofSeconds(30)  
);  
  
if (!locked) {  
    return "Already processing";  
}  
  
try {  
    processTransaction(txId);  
} finally {
```



```
unlockSafely(txId, randomToken);  
}
```

Tuy nhiên, distributed lock không phải cây đũa thần.

Vấn đề thực tế:

Lock hết hạn nhưng process chưa xong
Pod khác lấy lock và xử lý trùng
Redis failover có thể làm mất lock
Clock/network delay gây race condition
Unlock nhầm lock của process khác nếu không check token



Best practice:

Luôn dùng TTL
Luôn dùng random token
Unlock bằng Lua script để check token rồi delete
Không dùng lock thay thế cho idempotency
Lock chỉ là lớp bảo vệ phụ
DB unique constraint vẫn là lớp bảo vệ cuối



Trong banking, xử lý chống trùng thường phải có nhiều lớp:

Idempotency key
Redis lock
Database unique constraint
Transaction log
Message deduplication
Reconciliation



1.6 Redis dùng cho idempotency

Idempotency nghĩa là cùng một request gửi nhiều lần nhưng kết quả nghiệp vụ chỉ xảy ra một lần.

Ví dụ client gửi request chuyển tiền:

```
POST /transfers  
Idempotency-Key: abc-123
```

Server kiểm tra Redis:

idempotency:abc-123



Nếu chưa có:

Set PROCESSING
Xử lý giao dịch
Lưu kết quả SUCCESS/FAILED



Nếu có:

PROCESSING -> trả "đang xử lý"
SUCCESS -> trả lại kết quả cũ
FAILED -> tùy case cho retry hoặc trả lỗi cũ



Pseudo design:

Key: idem:{clientId}:{idempotencyKey}
Value:
{
 "status": "SUCCESS",
 "response": {...},
 "createdAt": "...",
 "transactionId": "..."
}
TTL: 24h hoặc theo policy



Điểm quan trọng: Redis giúp nhanh, nhưng kết quả cuối vẫn nên lưu DB. Nếu Redis mất dữ liệu mà DB vẫn có transaction thì hệ thống vẫn không được xử lý trùng.

1.7 Redis dùng cho rate limiting

Ví dụ giới hạn mỗi user chỉ được gọi API OTP 5 lần / phút.

Redis counter:

INCR otp:user:123
EXPIRE otp:user:123 60



Nếu count > 5 thì reject.

Cách tốt hơn là dùng Lua script để INCR và EXPIRE atomic.

Các thuật toán rate limit thường dùng:

Fixed Window
Sliding Window
Token Bucket
Leaky Bucket



Với API Gateway, Redis thường được dùng để chia sẻ rate limit giữa nhiều instance gateway.

1.8 Redis persistence: RDB và AOF

Redis là in-memory, nhưng vẫn có cơ chế ghi xuống disk.

Có 2 kiểu phổ biến:

RDB

Redis chụp snapshot dữ liệu tại một thời điểm.

Ưu điểm:

file nhỏ
restart nhanh
phù hợp backup



Nhược điểm:

có thể mất dữ liệu từ lần snapshot cuối đến lúc crash



AOF

Redis ghi log từng lệnh write.

Ưu điểm:

durability tốt hơn RDB
ít mất dữ liệu hơn



Nhược điểm:

file lớn hơn
ghi nhiều hơn
cần rewrite AOF



Trong production, chọn RDB, AOF hoặc cả hai tùy mục đích.

Nếu Redis chỉ làm cache, mất dữ liệu có thể chấp nhận được.

Nếu Redis lưu idempotency key hoặc lock state quan trọng, cần thiết kế kỹ hơn, nhưng vẫn không nên coi Redis là source of truth duy nhất.

1.9 Redis Cluster

Redis single node có giới hạn:

RAM giới hạn
CPU single-thread command processing giới hạn
một node chết là nguy hiểm nếu không có replica



Redis Cluster giải quyết bằng cách chia data ra nhiều primary node.

Ví dụ cluster 3 primary, 3 replica:

Primary A: hash slots 0 - 5460
Primary B: hash slots 5461 - 10922
Primary C: hash slots 10923 - 16383



Replica A
Replica B
Replica C

Redis Cluster dùng hash slot để quyết định key nằm ở node nào. Tổng số slot là 16,384.



Redis +1

Ví dụ:

user:1 -> slot 1200 -> Primary A
user:2 -> slot 9000 -> Primary B
order:5 -> slot 15000 -> Primary C



Khi client query key, Redis client cluster-aware sẽ biết key thuộc node nào.

1.10 Redis hash tag

Trong Redis Cluster, nếu một command cần nhiều key, các key đó thường phải nằm cùng slot.

Ví dụ:

```
MGET user:1:profile user:1:setting
```



Nếu 2 key nằm khác slot, Redis Cluster có thể báo lỗi cross-slot.

Dùng hash tag:

```
user:{1}:profile  
user:{1}:setting
```



Redis chỉ hash phần trong `{}` . Hai key này cùng slot vì cùng hash tag `{1}` .

Best practice:

```
Dùng hash tag cho dữ liệu cần thao tác multi-key  
Không lạm dụng làm hot slot  
Thiết kế key naming từ đầu
```



1.11 Redis Sentinel vs Redis Cluster

Redis Sentinel thường dùng cho high availability của Redis không cluster. Sentinel có thể giám sát Redis primary/replica và thực hiện failover khi primary lỗi.  Redis

So sánh:

Tiêu chí	Sentinel	Cluster
Mục tiêu chính	High availability	Sharding + availability
Chia dữ liệu	Không	Có
Scale RAM	Không tốt	Tốt hơn
Failover	Có	Có
Độ phức tạp	Thấp hơn	Cao hơn
Dùng khi	Redis nhỏ/vừa	Redis lớn, traffic cao

1.12 Quản lý Redis trong production

Một hệ thống Redis production cần quan tâm:



```
memory usage
eviction policy
slow commands
hot key
big key
connection pool
latency
replication lag
AOF rewrite
backup
failover
security
network
```

Eviction policy

Khi Redis đầy RAM, nó phải quyết định xóa key nào.

Một số policy phổ biến:



```
noeviction
allkeys-lru
volatile-lru
allkeys-lfu
volatile-ttl
```

Nếu Redis làm cache, thường dùng LRU/LFU.

Nếu Redis lưu lock/idempotency/rate limit, phải cẩn thận vì eviction có thể xóa key quan trọng.

Best practice:



```
Tách Redis cache và Redis coordination ra cluster khác nhau
Không để cache eviction ảnh hưởng idempotency/lock
Set maxmemory rõ ràng
Monitor memory fragmentation
```

1.13 Redis best practice trong dự án lớn

Một thiết kế Redis tốt thường có các nguyên tắc:



```
Redis không phải source of truth cho dữ liệu tài chính quan trọng
TTL phải rõ ràng
Key naming phải có convention
Không lưu object quá lớn
```

Không dùng KEYS trong production
 Dùng SCAN thay KEYS nếu cần duyệt
 Connection pool phải được cấu hình
 Command phải có timeout
 Không retry vô hạn khi Redis down
 Tách cache / lock / rate limit / session nếu workload khác nhau
 Monitor hit ratio, latency, memory, evicted keys

Ví dụ naming convention:

```

cache:user-profile:{userId}
lock:payment:{transactionId}
idem:transfer:{clientId}:{idempotencyKey}
rate:otp:{userId}
session:{sessionId}
  
```



2. Xử lý file trong ứng dụng Java

2.1 Vì sao xử lý file là vấn đề lớn?

Xử lý file nghe đơn giản, nhưng trong doanh nghiệp rất dễ gây lỗi production.

Ví dụ:

```

Upload file Excel 5MB
Import CSV 2GB
Generate report 10 triệu dòng
Download statement PDF
Process bank reconciliation file cuối ngày
Read log file
Parse file từ SFTP
  
```



Các lỗi thường gặp:

```

OutOfMemoryError
file upload treo request
disk đầy
file bị đọc dở
encoding sai
duplicate processing
partial write
security vulnerability
file name path traversal
slow I/O làm nghẽn thread
  
```



2.2 Xử lý file nhỏ

File nhỏ là file có thể đọc toàn bộ vào memory mà không nguy hiểm.

Ví dụ dưới vài MB.

```
String content = Files.readString(Path.of("data.txt"), StandardCharsets.UTF_8);
```

Hoặc:

```
byte[] bytes = Files.readAllBytes(path);
```

Dùng cho:

```
config nhỏ  
template nhỏ  
JSON nhỏ  
text nhỏ  
unit test fixture
```



Nhưng trong API production, không nên đọc upload file vào memory nếu không giới hạn size.

Bad practice:

```
byte[] data = multipartFile.getBytes();
```

Nếu user upload file 500MB, app có thể chết memory.

2.3 Xử lý file lớn: dùng streaming

Với file lớn, nguyên tắc là:

```
Không load toàn bộ file vào RAM.  
Đọc từng phần.  
Xử lý từng dòng hoặc từng block.  
Ghi từng phần.
```



Ví dụ đọc CSV lớn:


```
try (BufferedReader reader = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
    String line;
    while ((line = reader.readLine()) != null) {
        processLine(line);
    }
}
```

Với binary file:

```
try (InputStream in = Files.newInputStream(path);
    OutputStream out = Files.newOutputStream(target)) {

    byte[] buffer = new byte[8192];
    int bytesRead;

    while ((bytesRead = in.read(buffer)) != -1) {
        out.write(buffer, 0, bytesRead);
    }
}
```

Best practice:

Dùng try-with-resources
Dùng buffer
Không giữ toàn bộ file trong List
Validate file size
Validate file type
Set timeout
Ghi file tạm rồi rename atomic



2.4 Upload file trong Spring Boot

Với file nhỏ, có thể dùng `MultipartFile`.

```
@PostMapping("/upload")
public ResponseEntity<?> upload(@RequestParam MultipartFile file) {
    validate(file);
    storageService.save(file);
    return ResponseEntity.ok().build();
}
```

Nhưng với file lớn, không nên xử lý nặng ngay trong HTTP request.

Sai:

```
Client upload file 2GB
  -> API nhận file
  -> parse ngay trong request
  -> insert DB ngay
  -> request timeout
  -> client retry
  -> duplicate data
```



Đúng hơn:

```
Client upload file
  -> lưu file vào object storage / temp storage
  -> tạo job record trong DB
  -> return jobId
  -> batch/worker xử lý file async
  -> client check status
```



Flow production:

```
POST /imports
  -> validate metadata
  -> save file
  -> create import_job status=PENDING
  -> publish event
  -> return jobId
```



Worker

```
-> pick job
-> process streaming
-> update progress
-> mark SUCCESS/FAILED
```

2.5 Xử lý file CSV lớn

CSV tưởng đơn giản nhưng rất dễ lỗi:

```
comma trong quote
newline trong quote
encoding UTF-8/UTF-16
BOM
date format
decimal format
```



empty column
duplicate row
bad row

Không nên tự split bằng dấu phẩy:

```
String[] columns = line.split(",");
```

Vì dữ liệu này sẽ lỗi:

```
1,"Nguyen, Van A",1000
```

Nên dùng library như:

Apache Commons CSV
OpenCSV
uniVocity-parsers



Với file lớn, xử lý theo chunk:

Đọc 1000 dòng
Validate
Insert batch
Commit
Tiếp tục 1000 dòng



Không nên:

Đọc 10 triệu dòng vào List
Sau đó mới insert



2.6 Xử lý Excel lớn

Excel .xlsx có thể rất nặng. Apache POI bình thường có thể ăn rất nhiều RAM.

Với Excel lớn:

Dùng streaming API
Dùng SXSSFWorkbook để ghi file lớn
Dùng event-based reader nếu đọc cực lớn



Giới hạn số dòng
Chuyển sang CSV nếu business cho phép

Trong doanh nghiệp, nếu import hàng triệu dòng, CSV thường phù hợp hơn Excel.

2.7 Generate file lớn

Ví dụ export báo cáo giao dịch 5 triệu dòng.

Bad design:

```
List<Transaction> list = repository.findAll();  
generateExcel(list);
```

Vấn đề:

DB load quá nhiều
App memory tăng
Excel file quá lớn
Request timeout



Better design:

```
Client request export  
-> tạo export_job  
-> worker query DB theo page/cursor  
-> ghi streaming ra file  
-> upload lên S3/minio  
-> gửi link download
```



Pseudo:

```
long lastId = 0;  
  
while (true) {  
    List<Transaction> batch = repository.findNextBatch(lastId, 1000);  
  
    if (batch.isEmpty()) break;  
  
    for (Transaction tx : batch) {  
        writer.write(tx);  
        lastId = tx.getId();  
    }  
}
```

```
}  
}
```

Dùng cursor hoặc keyset pagination tốt hơn offset pagination cho dữ liệu lớn.

2.8 Atomic file write

Khi ghi file, không nên ghi thẳng vào file final.

Sai:

```
report.csv
```



Nếu app crash giữa chừng, người khác có thể đọc file dở.

Đúng:

```
report.csv.tmp  
ghi xong  
fsync nếu cần  
rename thành report.csv
```



Rename trên cùng filesystem thường atomic.

Flow:

```
write to temp file  
validate checksum  
rename temp -> final  
mark DB status SUCCESS
```



2.9 File security

File upload là nơi rất dễ bị tấn công.

Các lỗi phổ biến:

```
path traversal  
upload malware  
file extension giả  
file quá lớn  
zip bomb  
CSV injection  
Excel formula injection
```



```
overwrite file
public access sai
```

Ví dụ path traversal:

```
filename = ../../../../etc/passwd
```



Không bao giờ dùng trực tiếp filename từ user để tạo path.

Đúng:

```
String safeFileName = UUID.randomUUID() + ".csv";
Path target = uploadDir.resolve(safeFileName).normalize();
```

CSV injection:

```
=HYPERLINK("malicious")
+cmd|'/C calc'!A0
```

Khi export CSV cho Excel, các cell bắt đầu bằng =, +, -, @ cần escape nếu dữ liệu đến từ user.

2.10 Best practice xử lý file trong enterprise

```
Không xử lý file lớn trong HTTP request
Lưu metadata vào DB
Lưu binary vào object storage
Xử lý async bằng batch/worker
Có job status: PENDING, PROCESSING, SUCCESS, FAILED
Có retry policy
Có dead-letter/manual review
Có checksum
Có audit log
Có virus scan nếu file từ bên ngoài
Có limit size/type/row count
Có cleanup temp file
Có idempotency cho import
Có reconciliation sau xử lý
```



3. Multithread trong Java

3.1 Vì sao cần multithread?

Một chương trình Java bình thường chạy tuần tự:

```
Task A
Task B
Task C
```



Nếu mỗi task mất 1 giây, tổng 3 giây.

Multithread cho phép chạy đồng thời:

```
Thread 1 -> Task A
Thread 2 -> Task B
Thread 3 -> Task C
```



Nhưng multithread không tự động làm mọi thứ nhanh hơn. Nó chỉ giúp khi:

```
Có nhiều request đồng thời
Có I/O chờ database/network/file
Có task độc lập
Có CPU nhiều core
```



Nếu dùng sai, nó làm hệ thống khó debug hơn rất nhiều.

3.2 Thread truyền thống trong Java

Trước JDK 21, khi nói thread trong Java, chủ yếu là **platform thread**.

Platform thread thường map gần với OS thread.

```
Java Thread
  -> JVM
    -> OS Thread
```



OS thread đắt:

```
tốn memory stack
context switch tốn CPU
số lượng hữu hạn
tạo quá nhiều thread dễ chết hệ thống
```



Vì vậy Java enterprise lâu nay dùng thread pool.

3.3 Thread pool

Không tạo thread mới cho từng request/task. Thay vào đó, tạo sẵn một pool.

```
ExecutorService executor = Executors.newFixedThreadPool(100);
```

Flow:

Task đến

- > đưa vào queue
- > thread rảnh lấy ra xử lý
- > xử lý xong quay lại pool



Ví dụ hệ thống có pool 200 threads, 300 request đồng thời:

200 request được xử lý
100 request chờ trong queue



Nếu queue đầy:

reject
timeout
caller-runs
block



Tùy policy.

3.4 ThreadPoolExecutor quan trọng như thế nào?

Trong production, không nên dùng bữa:

```
Executors.newCachedThreadPool()
```

Vì nó có thể tạo rất nhiều thread.

Nên cấu hình rõ:

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(  
    20, // corePoolSize
```



```
100, // maximumPoolSize
60, TimeUnit.SECONDS, // keepAliveTime
new ArrayBlockingQueue<>(1000),
new ThreadPoolExecutor.CallerRunsPolicy()
);
```

Ý nghĩa:

corePoolSize: số thread giữ thường trực
maximumPoolSize: số thread tối đa
queue: nơi task chờ
rejection policy: xử lý khi quá tải



Thiết kế thread pool là thiết kế áp lực hệ thống.

Nếu queue quá dài:

request không bị reject ngay
nhưng latency tăng rất cao
user chờ lâu
timeout dây chuyền



Nếu queue quá ngắn:

reject sớm
bảo vệ hệ thống tốt hơn
nhưng user thấy lỗi nhiều hơn



Trong banking, thường ưu tiên **fail fast có kiểm soát** hơn là để hệ thống treo.

3.5 BlockingQueue

BlockingQueue là queue thread-safe dùng trong producer-consumer.

Ví dụ:

Producer tạo task
Consumer xử lý task



Nếu queue rỗng, consumer chờ.

Nếu queue đầy, producer có thể chờ hoặc bị reject.

Các loại phổ biến:

ArrayBlockingQueue
LinkedBlockingQueue
PriorityBlockingQueue
DelayQueue
SynchronousQueue



ArrayBlockingQueue

Có size cố định.

```
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(1000);
```

Ưu điểm:

kiểm soát memory
phù hợp production



LinkedBlockingQueue

Có thể rất lớn nếu không set capacity.

Nguy hiểm:

```
new LinkedBlockingQueue<>()
```

Vì queue gần như unbounded, task dồn vào có thể gây OutOfMemoryError.

SynchronousQueue

Không lưu task. Producer phải handoff trực tiếp cho consumer.

Hay dùng với cached thread pool.

3.6 Các vấn đề thường gặp trong multithread

Race condition

Hai thread cùng sửa một biến.

</> Java



Nhìn đơn giản nhưng thực chất là:

```
read count
increase
write count
```



Hai thread có thể đọc cùng giá trị rồi ghi đè nhau.

Cách xử lý:

```
synchronized
Lock
AtomicInteger
LongAdder
ConcurrentHashMap
Thiết kế immutable
```



Deadlock

Thread A giữ lock 1, chờ lock 2.

Thread B giữ lock 2, chờ lock 1.

```
Thread A: lock account A -> lock account B
Thread B: lock account B -> lock account A
```



Trong chuyển tiền, đây là lỗi kinh điển.

Cách xử lý:

```
Luôn lock theo thứ tự cố định
Giảm phạm vi lock
Dùng timeout lock
Tránh nested lock
```



Ví dụ lock theo accountId nhỏ trước:

```
Account first = accountA.id() < accountB.id() ? accountA : accountB;
Account second = accountA.id() < accountB.id() ? accountB : accountA;
```

Starvation

Một thread không bao giờ được chạy vì tài nguyên luôn bị thread khác chiếm.

Livelock

Thread không bị block, nhưng cứ phản ứng qua lại nên không tiến triển.

Thread leak

Tạo thread nhưng không shutdown.

```
ExecutorService executor = Executors.newFixedThreadPool(10);  
// quên executor.shutdown()
```

Trong app server, thread leak có thể làm app không shutdown sạch hoặc memory leak.

Blocking I/O làm nghẽn thread

Ví dụ mỗi request gọi database mất 2 giây. Nếu có 200 thread, tối đa 200 request đang chờ DB. Request 201 phải chờ queue.

Trong kiến trúc cũ, thread là tài nguyên khan hiếm. Vì vậy phải dùng:

```
connection pool  
timeout  
bulkhead  
circuit breaker  
async / reactive / virtual thread
```



3.7 JDK 21: Virtual Thread

JDK 21 chính thức đưa **virtual threads** vào Java Platform qua JEP 444. Virtual thread là lightweight thread giúp viết, bảo trì và debug ứng dụng concurrent high-throughput dễ hơn. 🚀 openjdk.org +1

Trước đây:

1 request blocking
-> giữ 1 platform thread



Với virtual thread:

1 request
-> chạy trên virtual thread
-> khi blocking I/O, virtual thread có thể unmount khỏi carrier thread
-> carrier thread rảnh để chạy virtual thread khác



Hình dung:

Virtual Thread 1
Virtual Thread 2
Virtual Thread 3
...
Virtual Thread 100000

chạy trên một số ít Platform Thread / Carrier Thread



Virtual thread rẻ hơn platform thread rất nhiều, nên có thể tạo rất nhiều virtual thread cho các task blocking I/O.

Ví dụ:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        executor.submit(() -> {
            callRemoteService();
            return null;
        });
    }
}
```

3.8 Virtual thread giải quyết vấn đề gì?

Nó đặc biệt phù hợp cho hệ thống có nhiều I/O blocking:

HTTP request
database call
REST call
file I/O
message processing



Trước JDK 21, nếu muốn scale I/O cao, có hai hướng:

Tăng thread pool lớn hơn
Dùng reactive programming như WebFlux



Reactive giúp tiết kiệm thread nhưng code khó đọc hơn.

Virtual thread cho phép giữ style code blocking dễ hiểu:

```
User user = userClient.getUser(id);  
Account account = accountClient.getAccount(id);  
return buildResponse(user, account);
```

Nhưng vẫn scale tốt hơn platform thread khi bị blocking I/O.

3.9 Virtual thread không giải quyết mọi thứ

Sai lầm phổ biến:

Có virtual thread rồi thì không cần thread pool, không cần limit, không cần timer



Sai.

Virtual thread rẻ, nhưng tài nguyên downstream không rẻ.

Ví dụ:

App tạo 100,000 virtual threads
Tất cả gọi DB
DB connection pool chỉ có 50 connections



Kết quả:

Virtual thread chờ connection
DB nghẽn
timeout
latency tăng



Vẫn cần:

DB connection pool
Rate limit
Bulkhead



Timeout
Circuit breaker
Backpressure

3.10 Virtual thread và synchronized

Một vấn đề cần biết: nếu virtual thread bị blocking trong một số vùng synchronized hoặc native call, nó có thể bị **pinning**, nghĩa là giữ carrier thread lâu hơn mong muốn.

Best practice:

Tránh synchronized block dài
Không giữ lock khi gọi I/O
Dùng ReentrantLock trong một số case
Đo bằng monitoring/JFR



3.11 Multithread best practice trong enterprise

Không dùng shared mutable state nếu không cần
Ưu tiên immutable object
Dùng concurrent collection đúng chỗ
Luôn set timeout khi gọi network/DB
Không dùng unbounded queue
Không tạo thread thủ công bừa bãi
Đặt tên thread rõ ràng
Có metrics cho active threads, queue size, rejected tasks
Tách thread pool theo loại workload
Không để task chậm chiếm pool của task quan trọng
Dùng virtual thread cho blocking I/O nếu runtime/framework hỗ trợ tốt
Không dùng virtual thread để chạy CPU-bound task khổng lồ mà không giới hạn



4. Batch job trong Java

4.1 Batch job là gì?

Batch job là chương trình xử lý dữ liệu theo lô, thường không phục vụ trực tiếp request realtime.

Ví dụ:



```
Chạy cuối ngày
Import file giao dịch
Đối soát ngân hàng
Tính phí
Tính lãi
Gửi email hàng loạt
Generate report
Sync dữ liệu giữa hệ thống
Clean dữ liệu cũ
Retry giao dịch lỗi
```

Trong banking, batch job cực kỳ quan trọng vì nhiều nghiệp vụ không chạy realtime hoàn toàn.

Ví dụ cuối ngày:



```
EOD - End of Day
-> khóa nhận giao dịch
-> tổng hợp giao dịch
-> tính phí
-> đối soát
-> sinh báo cáo
-> gửi file cho đối tác
```

4.2 Tại sao không xử lý tất cả bằng API realtime?

API realtime cần phản hồi nhanh.

Batch job xử lý những việc:



```
nặng
chậm
số lượng lớn
có thể chạy nền
cần retry
cần checkpoint
cần audit
không cần user chờ trực tiếp
```

Ví dụ import 10 triệu dòng giao dịch. Nếu xử lý trong HTTP request thì request sẽ timeout. Batch job phù hợp hơn.

4.3 Spring Batch

Trong Java enterprise, framework phổ biến là **Spring Batch**.

Spring Batch có mô hình:

Job

-> Step 1
-> Step 2
-> Step 3



Mỗi Step có thể dùng:

ItemReader
ItemProcessor
ItemWriter



Spring Batch hỗ trợ chunk-oriented processing, tức là đọc từng item, gom thành chunk, ghi chunk trong transaction boundary. Khi số item đọc đạt commit interval, Spring Batch gọi writer và commit transaction. [Home](#)

Flow:

```
read item 1
process item 1

read item 2
process item 2

...

đủ 1000 item
write 1000 item
commit
```



4.4 Chunk-oriented processing

Ví dụ:

```
@Bean
public Step importStep(JobRepository jobRepository,
    PlatformTransactionManager transactionManager) {
    return new StepBuilder("importStep", jobRepository)
        .<InputRow, TransactionEntity>chunk(1000, transactionManager)
        .reader(reader())
        .processor(processor())
    }
```

```
.writer(writer())  
.build();  
}
```

Ý nghĩa:

```
Đọc 1000 dòng  
Process 1000 dòng  
Write 1000 dòng  
Commit
```



Nếu lỗi ở dòng 550 trong chunk, transaction của chunk đó rollback. Các chunk trước đã commit thì không cần làm lại.

4.5 Restartability

Batch job production phải restart được.

Ví dụ job xử lý 10 triệu dòng, tới dòng 6 triệu thì server crash.

Không thể chạy lại từ đầu nếu dữ liệu không idempotent.

Cần checkpoint:

```
Đã xử lý đến file offset nào  
Đã xử lý đến line number nào  
Đã xử lý đến transactionId nào  
Step nào đã complete
```



Spring Batch lưu metadata như:

```
JOB_INSTANCE  
JOB_EXECUTION  
STEP_EXECUTION  
EXECUTION_CONTEXT
```



Spring Batch có cơ chế restart job; step đã COMPLETED thường sẽ được skip khi restart, trừ khi cấu hình cho chạy lại.  Home

4.6 Skip và retry

Trong file 1 triệu dòng, có thể 10 dòng lỗi format.

Không nên fail cả job nếu business cho phép skip.

Cấu hình:

```
retry lỗi tạm thời: DB timeout, network error
skip lỗi data xấu: invalid date, missing optional field
fail lỗi nghiêm trọng: schema sai, file sai format
```



Ví dụ policy:

```
Retry 3 lần với lỗi database connection
Skip tối đa 100 bad records
Nếu bad records > 100 thì fail job
```



4.7 Idempotency trong batch

Batch rất dễ xử lý trùng khi restart hoặc retry.

Ví dụ import file giao dịch:

```
Dòng 1-1000 đã insert DB
Job crash trước khi update status
Restart lại đọc lại 1-1000
Insert trùng
```



Cách xử lý:

```
Unique constraint theo business key
Upsert nếu phù hợp
Staging table
Process status
Idempotency key
File checksum
Transaction boundary rõ
```



Ví dụ:

```
CREATE UNIQUE INDEX uk_transaction_ref
ON transaction_import(file_id, transaction_ref);
```

Như vậy dù job retry, DB vẫn bảo vệ không insert trùng.

4.8 Batch kết hợp Jenkins để làm gì?

Jenkins không chỉ dùng để CI/CD. Jenkins cũng hay được dùng để trigger batch job thủ công hoặc theo lịch.

Ví dụ:

```
Chạy batch đối soát cuối ngày
Chạy job import file từ đối tác
Chạy script cleanup dữ liệu test
Chạy job generate report
Deploy xong trigger data migration
```



Nhưng trong production hiện đại, nếu batch là workload dài hạn, thường nên chạy bằng:

```
Kubernetes CronJob
Airflow
Control-M
Autosys
Argo Workflows
Spring Cloud Data Flow
```



Jenkins có thể dùng, nhưng cần cẩn thận vì Jenkins không phải batch scheduler chuyên dụng cho nghiệp vụ tài chính phức tạp.

4.9 Ví dụ Jenkins trigger Spring Batch

```
Jenkins job
-> checkout code hoặc dùng image có sẵn
-> chạy command:
    java -jar batch-app.jar --job.name=reconciliationJob --date=2026-06-25
```



Hoặc nếu chạy trên Kubernetes:

```
Jenkins
-> kubectl create job reconciliation-20260625 --from=cronjob/reconciliation
```



4.10 Vấn đề thường gặp với batch job

Job chạy quá lâu

Nguyên nhân:

```
chunk size quá nhỏ
query không có index
```



ghi DB từng row
không partition
network chậm
writer gọi API external quá nhiều

Cách xử lý:

batch insert
tăng chunk size hợp lý
partition theo date/range
parallel step
index DB
cursor pagination
tránh offset lớn



Duplicate data

Cách xử lý:

unique key
idempotent writer
staging table
job execution id
file checksum
dedup trước khi write



Memory tăng

Nguyên nhân:

đọc toàn bộ file vào List
Hibernate persistence context giữ quá nhiều entity
cache không clear
Excel load full memory



Cách xử lý:

streaming
chunk processing
entityManager.clear()
JDBC batch thay vì JPA nếu cực lớn



Job restart sai

Nguyên nhân:

reader không lưu state
writer không idempotent
step design sai
transaction boundary sai



Cách xử lý:

lưu checkpoint
thiết kế business key
kiểm tra restart scenario trong test



4.11 Batch job best practice

Mỗi job phải có jobId/jobExecutionId
Có status rõ: STARTED, COMPLETED, FAILED
Có audit log
Có metrics: processed count, skipped count, failed count
Có alert khi fail
Có restart strategy
Có idempotency
Có timeout
Có locking để tránh 2 job cùng chạy
Có input validation trước khi process
Có staging table cho import lớn
Có reconciliation report sau xử lý
Không xử lý batch nặng trong API request



5. Jenkins và CI/CD

5.1 Jenkins là gì?

Jenkins là automation server mã nguồn mở, viết bằng Java, dùng để tự động hóa build, test, deploy và nhiều workflow khác. Jenkins có hệ sinh thái plugin rất lớn và có thể dùng như CI server hoặc continuous delivery hub.  Jenkins +1

5.2 CI/CD là gì?

CI - Continuous Integration

Mỗi khi developer push code:

```
checkout code
compile
run unit test
run static analysis
build artifact
```



Mục tiêu:

```
phát hiện lỗi sớm
tránh merge code hỏng
đảm bảo main branch luôn build được
```



CD - Continuous Delivery / Deployment

Sau khi build/test xong:

```
package artifact
build Docker image
push registry
deploy dev/staging/prod
run smoke test
rollback nếu lỗi
```



Continuous Delivery nghĩa là luôn sẵn sàng deploy.

Continuous Deployment nghĩa là tự động deploy đến production nếu pass các gate.

Jenkins Pipeline mô tả toàn bộ quá trình từ version control đến release dưới dạng pipeline tự động.  Jenkins

5.3 Jenkinsfile

Jenkins Pipeline thường được viết trong file `Jenkinsfile` và commit cùng source code. Jenkins official docs gọi đây là pipeline-as-code, cho phép lưu pipeline như một phần của source control.  Jenkins

Ví dụ pipeline Java Spring Boot:

```
pipeline {
    agent any
```

```
tools {
  jdk 'jdk21'
  maven 'maven-3.9'
}

environment {
  IMAGE_NAME = 'registry.example.com/payment-service'
  IMAGE_TAG = "${env.BUILD_NUMBER}"
}

stages {
  stage('Checkout') {
    steps {
      checkout scm
    }
  }

  stage('Build') {
    steps {
      sh 'mvn clean package -DskipTests'
    }
  }

  stage('Unit Test') {
    steps {
      sh 'mvn test'
    }
  }

  stage('Static Analysis') {
    steps {
      sh 'mvn verify'
    }
  }

  stage('Build Docker Image') {
    steps {
      sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
    }
  }

  stage('Push Docker Image') {
    steps {
      sh 'docker push $IMAGE_NAME:$IMAGE_TAG'
    }
  }

  stage('Deploy to Dev') {
    steps {
      sh 'helm upgrade --install payment-service ./helm/payment-service --'
```



```
}

post {
    success {
        echo 'Pipeline success'
    }
    failure {
        echo 'Pipeline failed'
    }
}
}
```

5.4 Pipeline production nên có những stage nào?

Một pipeline nghiêm túc thường có:

- Checkout
- Compile
- Unit Test
- Static Code Analysis
- Dependency Vulnerability Scan
- Secret Scan
- Build Artifact
- Build Docker Image
- Image Scan
- Push Registry
- Deploy Dev
- Integration Test
- Deploy Staging
- Manual Approval
- Deploy Production
- Smoke Test
- Rollback nếu fail
- Notify Slack/Email



Với banking, thường có thêm:

- security gate
- change approval
- audit trail
- artifact signing
- deployment approval
- segregation of duties



5.5 Jenkins agent

Không nên chạy mọi job trên Jenkins controller.

Kiến trúc tốt:

```
Jenkins Controller
    -> quản lý job, UI, credentials
Jenkins Agents
    -> chạy build/test/deploy
```



Với Kubernetes:

```
Jenkins Controller
    -> tạo dynamic pod agent
        -> pod build Java
        -> pod build Docker
        -> pod deploy
```



Ưu điểm:

```
scale agent theo nhu cầu
môi trường build sạch
giảm tải controller
```



5.6 Credentials trong Jenkins

Không hardcode password/token trong Jenkinsfile.

Sai:

```
sh 'docker login -u admin -p mypassword'
```

Đúng:

```
withCredentials([usernamePassword(
    credentialsId: 'docker-registry-creds',
    usernameVariable: 'USERNAME',
    passwordVariable: 'PASSWORD'
)]) {
    sh 'echo $PASSWORD | docker login -u $USERNAME --password-stdin registry.example.'
}
```

Best practice:

Dùng Jenkins Credentials
Rotate secret
Không echo secret
Giới hạn quyền credential theo folder/job
Tách credential dev/staging/prod



5.7 CI/CD deployment strategy

Rolling update

Deploy từng pod mới, thay dần pod cũ.

Ưu điểm:

đơn giản
không downtime nếu readiness probe đúng



Nhược điểm:

version cũ và mới cùng chạy trong một lúc
cần backward compatibility



Blue-green deployment

Có 2 environment:

blue: version hiện tại
green: version mới



Test green xong chuyển traffic.

Ưu điểm:

rollback nhanh
rủi ro thấp



Nhược điểm:

tốn tài nguyên gấp đôi
database migration phức tạp



Canary deployment

Chỉ cho một phần nhỏ traffic vào version mới.

Ví dụ:

```
5% traffic -> v2  
95% traffic -> v1
```



Nếu ổn thì tăng dần.

Dùng tốt với:

```
Istio  
Linkerd  
Argo Rollouts  
NGINX Ingress  
API Gateway
```



5.8 Vấn đề thường gặp với Jenkins

```
Pipeline chậm  
Agent thiếu dependency  
Test flaky  
Credential lộ  
Build không reproducible  
Deploy nhầm environment  
Không rollback được  
Jenkins controller quá tải  
Plugin conflict  
Không backup Jenkins home
```



Best practice:

```
Pipeline as code  
Shared library cho pipeline chung  
Agent immutable  
Không build trên controller  
Pin version plugin quan trọng  
Backup Jenkins config  
Dùng artifact repository  
Dùng Docker image build environment cố định  
Tách CI và CD permission  
Có manual approval cho prod  
Có rollback stage
```



6. API Gateway

6.1 API Gateway là gì?

API Gateway là cổng vào trung tâm cho client gọi vào hệ thống backend.

Thay vì client gọi trực tiếp nhiều microservice:

```
Mobile app -> user-service  
Mobile app -> payment-service  
Mobile app -> account-service  
Mobile app -> notification-service
```



Client gọi API Gateway:

```
Mobile app  
  -> API Gateway  
      -> user-service  
      -> payment-service  
      -> account-service
```



Gateway giúp hệ thống có một lớp kiểm soát chung.

6.2 Mục đích của API Gateway

API Gateway thường xử lý:

```
Routing  
Authentication  
Authorization  
Rate limiting  
Request validation  
SSL termination  
Logging  
Tracing  
Load balancing  
Circuit breaker  
Request/response transformation  
API versioning  
CORS  
WAF integration
```



Ví dụ:

```
/api/users/**      -> user-service  
/api/payments/**   -> payment-service  
/api/accounts/**   -> account-service
```



6.3 Tại sao microservice cần API Gateway?

Nếu không có gateway:

Client phải biết địa chỉ từng service
Auth logic lặp lại ở nhiều service
Rate limit khó đồng bộ
Logging/tracing phân tán
Expose nội bộ ra ngoài nguy hiểm



Có gateway:

Client chỉ biết gateway
Service nội bộ được che giấu
Policy tập trung
Dễ quan sát traffic
Dễ version API



6.4 API Gateway không nên làm gì?

Gateway không nên biến thành "super service".

Không nên nhét quá nhiều business logic vào gateway.

Sai:

Gateway tính phí giao dịch
Gateway quyết định approve loan
Gateway xử lý workflow thanh toán



Đúng:

Gateway xác thực token
Gateway route request
Gateway rate limit
Gateway kiểm tra request size
Gateway thêm correlation id
Gateway reject request rõ ràng nếu vi phạm policy



Business logic nên nằm trong domain service.

6.5 Cơ chế routing

Ví dụ Spring Cloud Gateway:

```
spring:
  cloud:
    gateway:
      routes:
        - id: payment-service
          uri: lb://payment-service
          predicates:
            - Path=/api/payments/**
          filters:
            - StripPrefix=1
```

Flow:

```
Request /api/payments/transfer
-> Gateway match Path=/api/payments/**
-> route đến payment-service
```



6.6 Authentication và Authorization

Gateway thường verify JWT:

```
Client gửi Authorization: Bearer token
Gateway verify signature
Gateway check expiration
Gateway extract userId, roles, scopes
Gateway forward identity header xuống service
```



Nhưng service nội bộ vẫn nên verify authorization quan trọng.

Không nên tin gateway 100% nếu security yêu cầu cao.

Best practice:

```
Gateway xác thực token
Service kiểm tra quyền nghiệp vụ
Internal traffic dùng mTLS hoặc service mesh
Không cho client giả header userId
Gateway strip sensitive incoming headers
```



6.7 Rate limiting tại Gateway

Gateway là nơi tốt để chặn traffic xấu trước khi vào backend.

Ví dụ:

```
/user/login: 5 request/phút/IP  
/otp/send: 3 request/phút/user  
/payment/transfer: 10 request/phút/user
```



Redis thường được dùng để lưu counter rate limit giữa nhiều instance gateway.

```
Gateway 1  
Gateway 2  
Gateway 3  
-> cùng dùng Redis
```



6.8 Circuit breaker tại Gateway

Nếu payment-service đang lỗi, gateway không nên tiếp tục đẩy request vô hạn.

Circuit breaker có trạng thái:

```
Closed: gọi bình thường  
Open: chặn nhanh, trả lỗi/fallback  
Half-open: thử lại một lượng nhỏ request
```



Mục tiêu:

```
bảo vệ downstream  
fail fast  
tránh cascading failure
```



6.9 Timeout

Không có timeout là lỗi production cực nguy hiểm.

Ví dụ:

```
Gateway timeout: 2s  
Service A timeout đến Service B: 1s  
DB timeout: 500ms
```



Timeout phải thiết kế theo chuỗi.

Sai:


```
Client timeout 5s
Gateway timeout 60s
Service timeout 120s
DB timeout vô hạn
```



Khi đó request chết từ client nhưng backend vẫn xử lý, gây lãng phí và duplicate risk.

6.10 Observability tại Gateway

Gateway là điểm đầu vào nên phải log và trace tốt.

Mỗi request nên có:

```
correlationId
requestId
userId nếu có
clientId
path
method
status
latency
upstream service
error code
```



Gateway nên inject correlation id:

```
X-Correlation-Id: abc-123
```



Rồi tất cả service dùng chung id này để trace.

6.11 API Gateway best practice

```
Không nhét business logic nặng vào gateway
Luôn có timeout
Luôn có rate limit cho endpoint nhạy cảm
Validate request size
Chặn path không hợp lệ
Strip header nguy hiểm
Dùng centralized auth
Có request id/correlation id
Có metrics latency/error/rps
Có circuit breaker
Có canary route nếu cần
```



Không expose service nội bộ trực tiếp
Tách public gateway và internal gateway nếu hệ thống lớn

7. Scale hệ thống với Kubernetes

7.1 Kubernetes là gì?

Kubernetes là nền tảng orchestration cho container. Nó giúp chạy hệ thống phân tán resilient hơn bằng cách hỗ trợ scaling, failover, deployment pattern và quản lý workload.



Thay vì chạy app kiểu thủ công:

```
SSH vào server  
copy jar  
java -jar app.jar
```



Với Kubernetes:

```
Build Docker image  
Apply deployment  
Kubernetes tạo pod  
Kubernetes restart nếu pod chết  
Kubernetes scale thêm pod nếu cần  
Kubernetes expose service
```



7.2 Các khái niệm cốt lõi

Cluster

Một Kubernetes cluster gồm nhiều node.

```
Kubernetes Cluster  
-> Control Plane  
-> Worker Node 1  
-> Worker Node 2  
-> Worker Node 3
```



Node

Máy chủ vật lý hoặc VM chạy workload.

Pod

Đơn vị nhỏ nhất trong Kubernetes.

Một pod thường chứa một container app.

Pod

-> container payment-service



Pod có IP riêng, nhưng pod có thể chết và được tạo lại, nên không nên gọi trực tiếp pod IP.

Deployment

Deployment quản lý số lượng pod mong muốn.

Ví dụ:

```
replicas: 3
```

Nghĩa là Kubernetes cố giữ 3 pod chạy.

Nếu 1 pod chết:

```
desired = 3
```

```
actual = 2
```

Kubernetes tạo pod mới



Kubernetes Deployment cũng hỗ trợ rolling update và autoscaling cho Deployment khi HPA được bật.  Kubernetes

Service

Service cung cấp endpoint ổn định để gọi pod.

```
payment-service.default.svc.cluster.local
```



Dù pod chết và IP đổi, service vẫn route đến pod mới.

Ingress

Ingress expose HTTP/HTTPS từ ngoài vào cluster.

Internet

-> Ingress Controller

-> Service

-> Pod



ConfigMap

Lưu config không nhạy cảm.

```
APP_MODE=prod
LOG_LEVEL=INFO
```



Secret

Lưu thông tin nhạy cảm như password/token, nhưng vẫn cần kết hợp encryption và secret management tốt hơn trong production.

Namespace

Chia cluster theo môi trường/team/module:

```
dev
staging
prod
monitoring
jenkins
```



7.3 Kubernetes hoạt động theo reconciliation loop

Kubernetes hoạt động theo nguyên tắc:

```
Bạn khai báo trạng thái mong muốn
Kubernetes liên tục làm thực tế khớp với mong muốn
```



Ví dụ manifest nói:

```
replicas = 5
```



Nếu thực tế chỉ còn 3 pod:

```
Kubernetes tạo thêm 2 pod
```



Nếu có 6 pod:

```
Kubernetes xóa bớt 1 pod
```



Đây là tư duy rất quan trọng: Kubernetes không phải script chạy một lần, mà là control system luôn reconcile.

7.4 Ví dụ Deployment Java service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payment-service
  template:
    metadata:
      labels:
        app: payment-service
    spec:
      containers:
        - name: payment-service
          image: registry.example.com/payment-service:1.0.0
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
          resources:
            requests:
              cpu: "500m"
              memory: "512Mi"
            limits:
              cpu: "1"
              memory: "1Gi"
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
            initialDelaySeconds: 20
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 20
```

7.5 Readiness probe và liveness probe

Readiness probe

Trả lời câu hỏi:

Pod này đã sẵn sàng nhận traffic chưa?



Nếu readiness fail, Kubernetes không route traffic vào pod đó.

Dùng khi:

app đang start
chưa connect DB xong
cache warmup chưa xong
migration chưa xong



Liveness probe

Trả lời câu hỏi:

Pod này còn sống thật không?



Nếu liveness fail, Kubernetes restart pod.

Sai lầm nghiêm trọng:

Liveness probe check DB



Nếu DB chậm chờn, tất cả pod có thể bị restart hàng loạt, làm outage nặng hơn.

Best practice:

Liveness: kiểm tra app process sống
Readiness: kiểm tra khả năng phục vụ traffic
Startup probe: dùng cho app start lâu



7.6 Resource request và limit

Ví dụ:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
```

```
limits:  
  cpu: "1"  
  memory: "1Gi"
```

Ý nghĩa:

```
request: Kubernetes dùng để scheduling  
limit: giới hạn tối đa container được dùng
```



Vấn đề Java trên Kubernetes:

Nếu memory limit là 1Gi nhưng JVM heap cấu hình sai, app có thể bị OOMKilled.

Best practice:

```
Cấu hình JVM container-aware  
Set -XX:MaxRAMPercentage hợp lý  
Không để heap ăn hết memory container  
Chừa memory cho metaspace, thread stack, direct buffer  
Theo dõi OOMKilled
```



Ví dụ:

```
Container memory: 1Gi  
Heap nên khoảng 60–70%  
Còn lại cho non-heap/native/thread/direct buffer
```



7.7 Horizontal Pod Autoscaler - HPA

HPA tự tăng/giảm số pod dựa trên metrics.

Kubernetes HPA controller scale workload như Deployment hoặc StatefulSet qua scale subresource.  Kubernetes

Ví dụ:

```
apiVersion: autoscaling/v2  
kind: HorizontalPodAutoscaler  
metadata:  
  name: payment-service-hpa  
spec:  
  scaleTargetRef:  
    apiVersion: apps/v1  
    kind: Deployment
```

```
name: payment-service
minReplicas: 3
maxReplicas: 20
metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Ý nghĩa:

Nếu CPU trung bình vượt 70%, tăng pod
Nếu thấp hơn lâu, giảm pod



Nhưng CPU không phải lúc nào cũng là metric tốt.

Với service I/O-bound, CPU có thể thấp nhưng latency cao vì:

DB connection pool hết
Thread pool queue dài
External API chậm
Kafka lag cao



Production tốt nên scale theo:

CPU
memory
request per second
latency
queue depth
Kafka consumer lag
custom business metric



7.8 Cluster Autoscaler

HPA scale pod, nhưng nếu node hết chỗ thì pod mới không chạy được.

Cluster Autoscaler scale node.

HPA muốn tăng từ 10 pod lên 30 pod
Cluster hiện không đủ CPU/memory
Cluster Autoscaler thêm node
Pod được schedule lên node mới



7.9 VPA

Vertical Pod Autoscaler đề xuất hoặc tự điều chỉnh resource request/limit.

Dùng cẩn thận với production vì thay đổi resource có thể cần restart pod.

7.10 Scaling Java service không chỉ là tăng pod

Một lỗi rất phổ biến:

Traffic tăng
Tăng pod từ 5 lên 50
Nhưng DB connection pool mỗi pod = 50
Tổng connection = 2500
DB chết



Scale app mà không scale downstream là tự bắn vào chân.

Cần tính:

number of pods
thread pool size per pod
DB connection pool per pod
Redis connection pool
Kafka consumer partition
external API limit
rate limit



Ví dụ:

20 pods
mỗi pod Hikari maxPoolSize = 20
=> tối đa 400 DB connections
DB có chịu được không?



Nếu DB chỉ chịu 200 connections:

maxPoolSize mỗi pod phải giảm
hoặc giới hạn max pod
hoặc dùng PgBouncer/proxy
hoặc tối ưu query/cache



7.11 Kubernetes Deployment strategy

Rolling update:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

Ý nghĩa:

maxUnavailable: 0 -> không giảm capacity khi deploy
maxSurge: 1 -> thêm tối đa 1 pod mới trong lúc rollout



Best practice:

Readiness probe phải đúng
Graceful shutdown phải đúng
preStop hook nếu cần
terminationGracePeriodSeconds đủ dài
App phải stop nhận request trước khi shutdown



7.12 Graceful shutdown cho Spring Boot

Khi Kubernetes terminate pod:

Gửi SIGTERM
Chờ terminationGracePeriodSeconds
Nếu app chưa chết, gửi SIGKILL



Spring Boot nên bật graceful shutdown:

```
server:
  shutdown: graceful

spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s
```

Mục tiêu:

```
ngừng nhận request mới  
xử lý xong request đang chạy  
đóng connection sạch  
commit/rollback transaction
```



7.13 Stateful service trên Kubernetes

Không phải service nào cũng stateless.

Stateless service:

```
payment-api  
user-api  
notification-api
```



Stateful workload:

```
database  
Kafka  
Redis  
Elasticsearch
```



Stateful workload cần:

```
StatefulSet  
PersistentVolume  
Stable network identity  
Backup/restore  
Careful upgrade
```



Trong enterprise, nhiều team chọn managed service cho database/Redis/Kafka thay vì tự vận hành trên Kubernetes, vì vận hành stateful system rất khó.

7.14 ConfigMap và Secret best practice

Không bake config vào Docker image.

Sai:

```
image payment-service-prod  
image payment-service-dev
```



Đúng:

same image
different config



Dùng:

ConfigMap cho config thường
Secret cho secret
External secret manager cho production nghiêm túc



Ví dụ:

AWS Secrets Manager
HashiCorp Vault
GCP Secret Manager
Azure Key Vault



7.15 Observability trên Kubernetes

Không thể scale tốt nếu không đo.

Cần có:

Metrics
Logs
Tracing
Alerting
Dashboard



Stack phổ biến:

Prometheus
Grafana
Loki / ELK / OpenSearch
Jaeger / Tempo
OpenTelemetry
Alertmanager



Metrics quan trọng cho Java service:

RPS
latency p95/p99
error rate
CPU
memory
GC pause



```
heap usage
thread count
DB connection pool usage
Redis latency
Kafka lag
HTTP client latency
```

7.16 Vấn đề thường gặp khi scale với Kubernetes

CrashLoopBackOff

Pod start rồi crash liên tục.

Nguyên nhân:

```
config sai
secret thiếu
DB không connect được
app lỗi startup
OOM
port sai
```



ImagePullBackOff

Kubernetes không pull được image.

Nguyên nhân:

```
image tag sai
registry credential sai
image chưa push
network lỗi
```



OOMKilled

Container vượt memory limit.

Nguyên nhân:

```
heap quá lớn
memory leak
file load full memory
thread quá nhiều
direct buffer
metaspace
```



Pod pending

Không có node đủ resource hoặc node selector/toleration sai.

Readiness fail

App chạy nhưng không sẵn sàng nhận traffic.

HPA không scale

Thiếu metrics-server, request CPU chưa set, metric không có.

Scale lên nhưng vẫn chậm

Nguyên nhân thường ở downstream:

DB nghẽn
Redis hot key
Kafka partition ít
external API limit
lock contention
thread pool queue



7.17 Kubernetes best practice cho hệ thống lớn

App stateless càng nhiều càng tốt
Mỗi service có request/limit rõ
Readiness/liveness/startup probe đúng
Graceful shutdown
Không dùng latest tag
Dùng immutable image tag
Dùng Helm/Kustomize/GitOps
Tách namespace theo môi trường/team
RBAC tối thiểu quyền
NetworkPolicy nếu cần isolation
Secrets không hardcode
HPA dựa trên metric đúng
Monitor p95/p99 latency
Log có correlation id
Deploy rolling/canary/blue-green
Có rollback plan
Test load trước khi scale production



Một ví dụ end-to-end thực tế

Giả sử xây hệ thống chuyển tiền.

Request realtime

Client



- > API Gateway
 - > verify JWT
 - > rate limit bằng Redis
 - > route payment-service
 - > check idempotency key trong Redis/DB
 - > lock transactionId bằng Redis
 - > validate account
 - > call core banking
 - > save transaction DB
 - > publish event Kafka
 - > return response

Batch cuối ngày

Jenkins/K8s CronJob trigger reconciliation batch



- > đọc file từ SFTP
- > validate checksum
- > Spring Batch đọc file theo chunk
- > ghi staging table
- > đối soát với transaction DB
- > sinh report
- > upload report
- > gửi notification

CI/CD

Developer push code



- > Jenkins checkout
- > unit test
- > integration test
- > security scan
- > build Docker image
- > push registry
- > deploy Kubernetes staging
- > smoke test
- > manual approval
- > deploy production

Scale

Kubernetes HPA scale payment-service

Redis Cluster scale cache/rate limit

DB connection pool giới hạn theo pod count



Gateway rate limit bảo vệ backend

Monitoring theo latency/error/DB pool/Redis latency

Checklist senior khi thiết kế production

Nếu bạn đi phỏng vấn senior Java banking, có thể trả lời theo mindset này:

1. Không chỉ làm cho chạy, phải làm cho đúng khi retry/fail/scale.
2. Không chỉ scale app, phải scale cả dependency phía sau.
3. Redis nhanh nhưng không phải source of truth cho tiền.
4. Batch phải restartable, idempotent, observable.
5. Multithread phải có backpressure, timeout, limit.
6. Gateway là nơi bảo vệ backend, không phải nơi chứa business logic.
7. Jenkins pipeline phải repeatable, secure, rollback được.
8. Kubernetes không tự cứu app design tệ; app phải cloud-native.
9. Mọi thứ quan trọng phải có metrics, alert, audit.



Danh sách thuật ngữ / keyword quan trọng

Redis

Redis: In-memory data store, thường dùng làm cache, lock, counter, session, rate limit.

Cache-aside: App tự check cache; miss thì đọc DB rồi ghi lại cache.

TTL: Time To Live, thời gian sống của key trong Redis.

Cache stale: Cache chứa dữ liệu cũ so với database.

Cache penetration: Request liên tục vào key không tồn tại, làm DB bị đánh trực tiếp.

Cache breakdown / hot key: Một key cực hot hết hạn, nhiều request cùng rebuild cache.

Cache avalanche: Nhiều key expire cùng lúc hoặc Redis down, traffic đổ xuống DB.

Distributed lock: Lock dùng chung giữa nhiều service instance, thường dùng Redis SET NX PX .

Idempotency: Cùng request gửi nhiều lần nhưng nghiệp vụ chỉ xảy ra một lần.

Redis Cluster: Redis chạy nhiều node, tự chia data bằng hash slot.

Hash slot: Redis Cluster chia key vào 16,384 slot.

Hash tag: Cú pháp `{}` trong Redis key để ép nhiều key cùng slot.

Sentinel: Cơ chế high availability cho Redis không cluster.

Eviction policy: Chính sách Redis xóa key khi đầy memory.

RDB: Snapshot persistence của Redis.

AOF: Append-only file, Redis ghi log command để tăng durability.

File processing

Streaming: Đọc/ghi file từng phần, không load hết vào memory.

Buffer: Vùng nhớ tạm để đọc/ghi I/O hiệu quả hơn.

MultipartFile: Object Spring dùng để nhận file upload.

Atomic write: Ghi file tạm rồi rename thành file final để tránh file dở.

Checksum: Giá trị kiểm tra tính toàn vẹn file.

Path traversal: Tấn công dùng filename như `../../etc/passwd`.

CSV injection: Dữ liệu CSV chứa công thức độc hại khi mở bằng Excel.

Object storage: Storage dạng S3/MinIO/GCS, dùng lưu file lớn.

Keyset pagination: Phân trang theo key/id thay vì offset, tốt hơn cho dữ liệu lớn.

Multithread

Thread: Luồng thực thi độc lập trong chương trình.

Platform thread: Thread truyền thống map gần với OS thread.

Virtual thread: Lightweight thread trong JDK 21, phù hợp blocking I/O high concurrency.

Carrier thread: Platform thread chạy virtual thread bên dưới.

Thread pool: Tập thread dùng lại để xử lý task.

BlockingQueue: Queue thread-safe, hỗ trợ chờ khi rỗng/đầy.

Race condition: Nhiều thread truy cập/sửa dữ liệu gây kết quả sai.

Deadlock: Các thread chờ lock lẫn nhau mãi mãi.

Starvation: Thread không được cấp tài nguyên để chạy.

Livelock: Thread vẫn chạy nhưng không tiến triển.

Thread leak: Thread tạo ra nhưng không được shutdown/thu hồi.

Backpressure: Cơ chế làm chậm hoặc reject input khi hệ thống quá tải.

Bulkhead: Tách tài nguyên để lỗi ở một phần không kéo chết toàn hệ thống.

Pinning: Virtual thread bị giữ carrier thread do blocking trong vùng không thể unmount tốt.

Batch job

Batch job: Job xử lý dữ liệu theo lô, thường chạy nền hoặc theo lịch.

Spring Batch: Framework Java phổ biến để xây batch job.

Job: Toàn bộ batch process.

Step: Một giai đoạn trong job.

ItemReader: Thành phần đọc dữ liệu.

ItemProcessor: Thành phần xử lý/chuyển đổi dữ liệu.

ItemWriter: Thành phần ghi dữ liệu.

Chunk: Nhóm item được xử lý và commit cùng nhau.

Commit interval: Số item trong một chunk trước khi commit.

Restartability: Khả năng chạy tiếp batch từ điểm lỗi.

Skip policy: Cho phép bỏ qua một số lỗi dữ liệu.

Retry policy: Cho phép thử lại với lỗi tạm thời.

Staging table: Bảng trung gian để import/validate dữ liệu trước khi đưa vào bảng chính.

EOD: End of Day, xử lý cuối ngày trong banking.

Reconciliation: Đối soát dữ liệu giữa các hệ thống.

Jenkins / CI/CD

Jenkins: Automation server dùng build, test, deploy.

CI: Continuous Integration, tự động build/test khi code thay đổi.

CD: Continuous Delivery/Deployment, tự động hóa release/deploy.

Jenkinsfile: File định nghĩa pipeline dưới dạng code.

Pipeline as Code: Lưu pipeline trong source control.

Agent: Máy/pod chạy job Jenkins.

Controller: Jenkins server chính quản lý job/UI/config.

Artifact: Kết quả build như .jar , Docker image.

Registry: Nơi lưu Docker image.

Static analysis: Phân tích code để tìm bug/security/code smell.

Blue-green deployment: Deploy version mới ở môi trường song song rồi switch traffic.

Canary deployment: Cho một phần nhỏ traffic vào version mới trước.

Rolling update: Thay pod cũ bằng pod mới từ từ.

API Gateway

API Gateway: Cổng vào của hệ thống microservice.

Routing: Điều hướng request đến service phù hợp.

Authentication: Xác thực người gọi là ai.

Authorization: Kiểm tra người gọi được phép làm gì.

Rate limiting: Giới hạn số request theo user/IP/client.

Circuit breaker: Ngắt tạm thời gọi đến service lỗi để tránh cascading failure.

Timeout: Giới hạn thời gian chờ.

Correlation ID: ID theo request để trace qua nhiều service.

CORS: Cơ chế browser kiểm soát cross-origin request.

SSL termination: Gateway xử lý HTTPS rồi forward nội bộ.

Request transformation: Biến đổi request/response nếu cần.

Kubernetes

Kubernetes: Nền tảng orchestration container.

Cluster: Tập node chạy Kubernetes.

Control Plane: Thành phần điều khiển cluster.

Node: Máy chạy workload.

Pod: Đơn vị deploy nhỏ nhất.

Deployment: Quản lý replica và rollout của pod.

Replica: Số bản sao pod.

Service: Endpoint ổn định để gọi pod.

Ingress: Lớp expose HTTP/HTTPS từ ngoài vào cluster.

ConfigMap: Lưu config không nhạy cảm.

Secret: Lưu secret/password/token.

Namespace: Không gian tách resource.

HPA: Horizontal Pod Autoscaler, tự scale số pod.

VPA: Vertical Pod Autoscaler, điều chỉnh resource request/limit.

Cluster Autoscaler: Tự tăng/giảm node.

Readiness probe: Kiểm tra pod sẵn sàng nhận traffic chưa.

Liveness probe: Kiểm tra pod còn sống không.

Startup probe: Kiểm tra app start lâu.

OOMKilled: Container bị kill vì vượt memory limit.

CrashLoopBackOff: Pod crash liên tục.

ImagePullBackOff: Không pull được Docker image.

Helm: Package manager cho Kubernetes manifest.

GitOps: Quản lý deploy bằng Git làm source of truth.

Service Mesh: Lớp hạ tầng như Istio/Linkerd để quản lý traffic, mTLS, observability giữa service.

    ...   Nguồn

trong các vấn đề trên, còn những chỗ nào bạn cảm thấy tôi đọc vào chắc chắn sẽ khó hiểu và chưa chi tiết lắm thì hãy xem lại và giải thích lại 1 cách chi tiết hơn, đừng mang tính liệt kê, hãy đi sâu vào thực tế vấn đề cần giải quyết.

ngoài ra về orchestrator để quản lý transaction phân tán giữa các service trong kafka, hãy giải thích chi tiết cận kề từng chút một về cách nó hoạt động, dùng gì quản lý, vấn đề hay gặp phải, hướng giải quyết, cách triển khai orchestrator

Mình sẽ sửa lại theo đúng chỗ “dễ đọc nhưng khó thấm”: không nhắc lại định nghĩa quá nhiều, mà bóc từng vấn đề production hay gây hiểu nhầm. Phần orchestrator + Kafka mình sẽ đi sâu nhất vì đây là phần dễ trả lời sai trong phỏng vấn senior.

Đã suy nghĩ trong 1m 29s >

Đúng, ở bàn trước có nhiều phần “đọc thì hiểu ý chính”, nhưng nếu đi phỏng vấn senior hoặc thiết kế hệ thống thật thì vẫn còn vài chỗ dễ hiểu lầm. Mình sẽ bóc lại các điểm đó trước, sau đó đi rất sâu vào phần **orchestrator quản lý transaction phân tán giữa các service dùng Kafka**.

I. Những chỗ chắc chắn dễ hiểu lầm nếu chỉ đọc bản trước

1. Redis: dễ nhầm giữa “nhanh” và “đúng”

Redis thường được giới thiệu là cache, distributed lock, rate limit, session store. Nhưng trong hệ thống lớn, vấn đề không nằm ở câu hỏi “Redis dùng để làm gì?”, mà nằm ở câu hỏi:

mất key, hết TTL, failover, hoặc bị duplicate request thì dữ liệu nghiệp vụ có c 

Đây là điểm rất quan trọng. Redis **không nên là nơi quyết định sự thật cuối cùng** cho nghiệp vụ quan trọng như tiền, sổ dư, trạng thái giao dịch, hạn mức, ledger. Redis có thể giúp hệ thống nhanh hơn, giảm tải DB, chặn request trùng sớm hơn, nhưng lớp bảo vệ cuối cùng vẫn nên nằm ở database, unique constraint, transaction log, ledger, hoặc reconciliation.

Ví dụ, bạn dùng Redis lock để chống xử lý trùng giao dịch:

```
SET lock:payment:TX123 abc NX PX 30000
```



Nghe có vẻ chắc chắn. Nhưng giả sử xử lý giao dịch mất 40 giây, còn lock chỉ sống 30 giây. Sau 30 giây, lock hết hạn. Một pod khác nhận retry request và lấy được lock mới. Vậy là vẫn có khả năng xử lý trùng.

Cho nên trong thực tế, Redis lock chỉ là **lớp giảm xác suất lỗi**, không phải **bằng chứng nghiệp vụ** rằng giao dịch chỉ xử lý một lần.

Thiết kế đúng thường là:

```
Client gửi idempotencyKey
  |
  v
Redis check nhanh xem request này đang xử lý chưa
  |
  v
Database có unique constraint trên idempotencyKey / transactionRef
  |
  v
Transaction log lưu trạng thái cuối cùng
  |
  v
Nếu có lỗi vẫn reconciliation được
```



Redis giúp bạn tránh việc nhiều request cùng lúc đập vào DB. Nhưng DB mới là lớp xác nhận cuối:

```
CREATE UNIQUE INDEX uk_payment_idempotency
ON payment_request(client_id, idempotency_key);
```

Nói cách khác:

Redis bảo vệ performance.
Database bảo vệ correctness.



Trong phỏng vấn senior, nếu bạn nói “dùng Redis lock để đảm bảo không duplicate transaction” thì câu trả lời chưa đủ. Câu tốt hơn là:

Em dùng Redis lock để giảm concurrent duplicate processing, nhưng không tin Rec 

2. Xử lý file: khó không phải là đọc file, mà là kiểm soát “vòng đời xử lý file”

Nhiều người nghĩ xử lý file trong Java là chọn `Files.readAllBytes`, `BufferedReader`, hay `InputStream`. Nhưng trong doanh nghiệp, vấn đề lớn hơn là:

File đến từ đâu?
File có hợp lệ không?
File có đang được upload dở không?
File xử lý được một nửa thì app chết thì sao?
File bị xử lý lại thì có duplicate data không?
File lỗi 3 dòng trong 1 triệu dòng thì fail hết hay skip?
File xử lý xong thì audit thế nào?



Với file nhỏ, bạn có thể đọc toàn bộ vào RAM. Nhưng với file lớn, nếu bạn chỉ nói “dùng stream” thì vẫn chưa đủ. Vì stream chỉ giải quyết memory, chưa giải quyết transaction, retry, duplicate, checkpoint.

Ví dụ import file giao dịch 2GB:

Dòng 1 – 1000: insert DB thành công
Dòng 1001 – 2000: insert DB thành công
Dòng 2001 – 3000: đang xử lý thì app crash



Khi restart, nếu app đọc lại từ đầu mà không có idempotency thì dữ liệu dòng 1 - 2000 có thể bị insert trùng.

Thiết kế thực tế nên có bảng job:

```
file_import_job
- id
- file_name
- checksum
- status: PENDING / PROCESSING / SUCCESS / FAILED
- total_rows
- processed_rows
```



- failed_rows
- created_at
- updated_at

Có bảng staging:

```
transaction_import_staging
- job_id
- line_number
- transaction_ref
- raw_payload
- validation_status
- error_message
```



Có unique constraint:

```
CREATE UNIQUE INDEX uk_import_line
ON transaction_import_staging(job_id, line_number);

CREATE UNIQUE INDEX uk_transaction_ref
ON transaction_import_staging(transaction_ref);
```

Lúc đó, file processing không còn là “đọc file” nữa. Nó trở thành một pipeline:

```
Upload file
-> lưu file
-> tạo job
-> validate checksum
-> đọc từng chunk
-> ghi staging
-> validate data
-> move từ staging sang bảng chính
-> sinh report lỗi
-> update job status
```



Trong banking, import file mà không có staging table là rất nguy hiểm, vì bạn mất khả năng audit, rollback logic, và giải thích vì sao dòng nào lỗi.

3. Multithread: khó không phải là tạo nhiều thread, mà là kiểm soát áp lực

Người mới hay nghĩ:

Muốn nhanh hơn thì tăng thread.



Nhưng senior phải nghĩ ngược lại:

Mỗi thread đang gây áp lực lên tài nguyên nào?



Một request có thể dùng:

1 thread ứng dụng
1 DB connection
1 Redis connection
1 HTTP connection đến service khác
memory buffer
CPU



Nếu bạn tăng thread pool từ 100 lên 1000, chưa chắc hệ thống nhanh hơn. Có thể bạn chỉ làm cho 1000 request cùng lúc tranh nhau 50 DB connections. Kết quả là thread chờ DB connection, queue dài, latency tăng, timeout dây chuyền.

Với `BlockingQueue`, điểm dễ hiểu lầm là queue không làm hệ thống khỏe hơn. Queue chỉ là chỗ chứa "nợ xử lý".

Ví dụ:

Thread pool: 100 threads
Queue: 100,000 tasks



Nhìn thì hệ thống không reject request. Nhưng thực tế task thứ 90,000 có thể phải chờ rất lâu mới được chạy. Đến lúc chạy thì client đã timeout. Đây là kiểu lỗi rất hay gặp: hệ thống không chết ngay, nhưng latency tăng dần rồi chết dây chuyền.

Thiết kế tốt hơn thường là:

Queue có giới hạn
Timeout ngắn
Reject sớm khi quá tải
Có retry có kiểm soát
Có circuit breaker
Có bulkhead tách pool cho từng loại workload



Với JDK 21 virtual thread, điểm dễ nhầm là tưởng virtual thread thay thế mọi thứ. Không phải. Virtual thread giúp rẻ hơn khi chờ I/O, nhưng nó không làm DB nhanh hơn, không làm API external chịu tải hơn, không làm Kafka partition tăng lên.

Ví dụ:

100,000 virtual threads gọi DB
DB pool chỉ có 50 connections



Kết quả vẫn nghẽn ở DB pool. Virtual thread chỉ làm chi phí chờ thấp hơn, không xóa bỏ bottleneck.

Cách nói senior hơn là:

Virtual thread giúp đơn giản hóa mô hình lập trình blocking I/O và giảm chi phí:



4. Batch job: khó không phải là chạy job, mà là chạy lại sau khi fail mà không sai dữ liệu

Batch job production phải trả lời được câu hỏi:

Nếu job chết giữa chừng, chạy lại có an toàn không?



Nếu câu trả lời là "không chắc", batch đó chưa đạt production-grade.

Một batch tốt phải có:

checkpoint
restartability
idempotent writer
skip policy
retry policy
audit log
job status
error report
reconciliation



Ví dụ job tính phí cuối ngày:

Tính phí cho 1 triệu account
Tới account thứ 600,000 thì DB timeout
Job fail



Khi chạy lại, hệ thống không được tính phí lại 600,000 account đầu. Cách làm đúng là lưu dấu vết nghiệp vụ:

fee_charge
- account_id
- fee_date



- amount
- status

Và unique constraint:

```
CREATE UNIQUE INDEX uk_fee_once_per_day  
ON fee_charge(account_id, fee_date);
```

Như vậy, dù batch retry, DB vẫn bảo vệ không tính phí trùng.

Điểm quan trọng: Jenkins có thể trigger batch, nhưng Jenkins không nên là nơi lưu trạng thái nghiệp vụ của batch. Trạng thái batch phải nằm trong DB hoặc batch framework như Spring Batch JobRepository. Jenkins chỉ nên là người bấm nút, lên lịch, hoặc chạy command.

5. Jenkins / CI/CD: khó không phải là viết pipeline, mà là kiểm soát artifact và rollback

Nhiều pipeline nhìn rất đẹp:

```
build  
test  
deploy
```



Nhưng production cần trả lời:

```
Artifact nào đang chạy ở production?  
Ai approve?  
Config nào được dùng?  
Deploy fail thì rollback về image nào?  
DB migration có backward compatible không?  
Secret có bị in ra log không?
```



Một pipeline senior không chỉ build code. Nó phải tạo ra một release có thể trace được:

```
Git commit SHA  
Build number  
Docker image tag  
Test result  
Security scan result  
Approval record  
Deployment environment  
Rollback version
```



Nếu deploy production bằng tag `latest`, đó là lỗi nghiêm trọng. Vì khi có sự cố, bạn không biết chính xác production đang chạy binary nào.

Tốt hơn:

```
payment-service:2026-06-25-1530-a8f31c2
payment-service:git-a8f31c2
```



CI/CD thực chất là kiểm soát rủi ro thay đổi. Không phải chỉ là tự động hóa lệnh deploy.

6. API Gateway: khó không phải là routing, mà là boundary của responsibility

API Gateway rất dễ bị biến thành “cục business logic khổng lồ”. Ban đầu chỉ route request. Sau đó thêm validate. Rồi thêm mapping. Rồi thêm logic tính phí. Rồi thêm rule đặc biệt. Cuối cùng gateway thành monolith trá hình.

Gateway nên xử lý logic hạ tầng:

```
authentication
authorization cơ bản
rate limit
routing
timeout
CORS
request size limit
correlation id
logging
circuit breaker
```



Gateway không nên xử lý logic domain kiểu:

```
tính phí giao dịch
quyết định approve loan
tính số dư
quyết định trạng thái thanh toán
```



Vì business logic đó cần nằm trong service sở hữu domain và database của nó.

Một điểm rất thực tế là timeout budget. Ví dụ client timeout 5 giây. Nếu gateway timeout 30 giây, service timeout 60 giây, DB timeout vô hạn thì khi client bỏ cuộc, backend vẫn còn xử lý. Điều này tạo ra duplicate, retry storm và tài nguyên bị giữ vô ích.

Timeout nên đi từ ngoài vào trong:

```
Client timeout: 5s
Gateway timeout: 4s
Service A gọi Service B: 2s
DB query timeout: 1s
```



7. Kubernetes: khó không phải là scale pod, mà là scale toàn bộ chuỗi phụ thuộc

Một lỗi rất hay gặp:

```
Traffic tăng
Tăng pod từ 5 lên 50
```



Nhìn thì đúng. Nhưng nếu mỗi pod có Hikari pool 20 connections:

```
5 pods   x 20 = 100 DB connections
50 pods x 20 = 1000 DB connections
```



DB có thể chết trước khi app kịp scale.

Cho nên scale Kubernetes phải tính theo chuỗi:

```
Ingress capacity
Gateway capacity
Service pod count
Thread / virtual thread concurrency
DB connection pool
Redis connection pool
Kafka partition count
External API limit
CPU / memory node
```



Kubernetes giúp bạn chạy nhiều bản sao. Nhưng nó không tự làm application đúng hơn. Nếu app không idempotent, không graceful shutdown, không timeout, không readiness đúng, thì scale lên chỉ làm lỗi xảy ra nhanh hơn.

II. Phần quan trọng nhất: Orchestrator quản lý transaction phân tán giữa các service dùng Kafka

Bây giờ mình đi sâu vào phần này.

Đầu tiên phải sửa một hiểu lầm rất lớn:

Kafka không phải transaction manager cho toàn bộ microservice.



Kafka có transaction của Kafka, nhưng transaction đó chủ yếu giải quyết atomicity trong phạm vi Kafka: producer ghi nhiều record, nhiều partition, và trong một số mô hình có thể commit offset cùng với output record. Kafka hỗ trợ các mức delivery như at-most-once, at-least-once, exactly-once; với exactly-once, producer idempotence, transaction, và consumer isolation level phải phối hợp đúng với nhau. Nhưng nếu service của bạn vừa update database vừa publish Kafka event, Kafka transaction một mình không tự bảo đảm atomic transaction giữa database và Kafka. [Confluent Docu...](#)

Vấn đề thật sự là:

Service A có database riêng.
Service B có database riêng.
Service C có database riêng.
Một nghiệp vụ cần đi qua cả A, B, C.
Làm sao đảm bảo toàn bộ business process cuối cùng nhất quán?



Trong microservice, mỗi service thường sở hữu database riêng. Vì vậy bạn không thể đơn giản dùng một local ACID transaction bao quanh tất cả service. Saga pattern sinh ra để xử lý vấn đề này: một business transaction lớn được tách thành chuỗi local transaction; mỗi local transaction update database của service đó và phát event/command để bước tiếp theo chạy. Nếu một bước thất bại vì lý do nghiệp vụ, saga chạy các bước compensation để "hoàn tác về mặt nghiệp vụ" những bước đã thành công trước đó. [microservices.io](#)

1. Vấn đề gốc: tại sao distributed transaction khó?

Giả sử có flow thanh toán:

1. Payment Service tạo payment
2. Account Service giữ tiền
3. Fraud Service kiểm tra rủi ro
4. Ledger Service ghi bút toán
5. Notification Service gửi thông báo



Mỗi service có database riêng:

payment_db
account_db
fraud_db



```
ledger_db  
notification_db
```

Bạn muốn tất cả cùng thành công hoặc cùng thất bại như một transaction duy nhất.

Trong monolith, bạn có thể làm:

```
@Transactional  
public void pay() {  
    paymentRepository.save(...);  
    accountRepository.reserve(...);  
    ledgerRepository.create(...);  
}
```

Nhưng trong microservice, bạn không thể @Transactional qua nhiều service/database như vậy một cách đơn giản.

Nếu bạn làm kiểu gọi tuần tự:

```
Payment Service gọi Account Service -> thành công  
Payment Service gọi Ledger Service -> timeout  
Payment Service không biết Ledger thật sự fail hay đã xử lý rồi
```

Lúc này hệ thống rơi vào trạng thái mơ hồ:

```
Account đã giữ tiền?  
Ledger đã ghi chưa?  
Payment nên SUCCESS hay FAILED?  
Có cần release tiền không?  
Nếu retry thì có ghi trùng ledger không?
```

Đây là lý do bạn cần orchestrator hoặc saga.

2. Saga có 2 kiểu: choreography và orchestration

2.1 Choreography

Choreography nghĩa là không có service trung tâm điều phối. Mỗi service nghe event rồi tự quyết định bước tiếp theo.

Ví dụ:

```
PaymentCreated event  
-> Account Service nghe, reserve money
```

-> publish MoneyReserved event

MoneyReserved event

-> Fraud Service nghe, check fraud

-> publish FraudPassed event

FraudPassed event

-> Ledger Service nghe, create ledger

-> publish LedgerCreated event

Ưu điểm là ít coupling trực tiếp. Service chỉ biết event, không biết ai gọi ai.

Nhưng nhược điểm production rất lớn:

Flow nghiệp vụ bị rải khắp nơi
 Khó biết payment đang ở bước nào
 Khó debug khi fail giữa chừng
 Khó xử lý timeout
 Khó thay đổi thứ tự bước
 Khó nhìn toàn bộ quy trình
 Compensation logic dễ rối



Choreography phù hợp với flow đơn giản, ít bước, ít compensation.

2.2 Orchestration

Orchestration nghĩa là có một thành phần trung tâm gọi là **orchestrator** quản lý state machine của saga.

Orchestrator sẽ nói rõ:

Bước 1: Account Service reserve money
 Bước 2: Fraud Service check fraud
 Bước 3: Ledger Service create ledger
 Bước 4: Payment Service mark success
 Nếu bước 3 fail: chạy compensation bước 1



Flow:

Payment API

-> tạo saga
 -> Orchestrator gửi command ReserveMoney
 -> Account Service xử lý
 -> gửi reply MoneyReserved
 -> Orchestrator nhận reply
 -> gửi command CheckFraud
 -> Fraud Service xử lý




```

-> gửi reply FraudPassed
-> Orchestrator nhận reply
-> gửi command CreateLedger
    -> Ledger Service xử lý
    -> gửi reply LedgerCreated
-> Orchestrator mark saga SUCCESS

```

Điểm hay của orchestrator là toàn bộ workflow nằm ở một nơi. Khi có lỗi, bạn biết saga đang ở step nào, chờ service nào, đã chạy compensation nào.

Microservices.io cũng mô tả rõ orchestration-based saga là kiểu orchestrator gửi command cho participant thực hiện local transaction, rồi nhận reply để quyết định approve/reject hoặc chạy bước tiếp theo. microservices.io

3. Orchestrator thật sự quản lý cái gì?

Orchestrator không quản lý transaction kiểu database lock toàn hệ thống. Nó quản lý **business transaction state**.

Nó cần biết:

Saga này là gì?
 Business key là gì?
 Đang ở bước nào?
 Bước nào đã thành công?
 Bước nào đang chờ reply?
 Bước nào cần retry?
 Bước nào cần compensation?
 Saga đã timeout chưa?
 Saga cuối cùng SUCCESS hay FAILED?



Vì vậy orchestrator cần một state store, thường là database.

Ví dụ bảng `saga_instance` :

```

CREATE TABLE saga_instance (
  id          UUID PRIMARY KEY,
  saga_type   VARCHAR(100) NOT NULL,
  business_key VARCHAR(200) NOT NULL,
  status      VARCHAR(50) NOT NULL,
  current_step VARCHAR(100),
  version     BIGINT NOT NULL,
  payload     JSONB,
  created_at  TIMESTAMP NOT NULL,
  updated_at  TIMESTAMP NOT NULL
)

```

```
);
```

```
CREATE UNIQUE INDEX uk_saga_business
ON saga_instance(saga_type, business_key);
```

Ví dụ bảng `saga_step` :

```
CREATE TABLE saga_step (
    id                UUID PRIMARY KEY,
    saga_id           UUID NOT NULL,
    step_name         VARCHAR(100) NOT NULL,
    status            VARCHAR(50) NOT NULL,
    command_id        UUID,
    reply_event_id    UUID,
    retry_count       INT NOT NULL DEFAULT 0,
    deadline_at       TIMESTAMP,
    error_code        VARCHAR(100),
    error_message     TEXT,
    created_at        TIMESTAMP NOT NULL,
    updated_at        TIMESTAMP NOT NULL
);
```

```
CREATE UNIQUE INDEX uk_saga_step
ON saga_step(saga_id, step_name);
```

Trạng thái có thể như:

```
STARTED
RESERVE_MONEY_PENDING
MONEY_RESERVED
FRAUD_CHECK_PENDING
FRAUD_PASSED
LEDGER_PENDING
COMPLETED
COMPENSATING
COMPENSATED
FAILED
```



Nhìn kỹ thì orchestrator là một **state machine**.

4. Kafka đóng vai trò gì trong orchestrator?

Kafka không phải nơi lưu state chính của saga. Kafka là message backbone để orchestrator và các service giao tiếp bất đồng bộ.

Orchestrator gửi command qua Kafka:

```
payment.saga.command.account
payment.saga.command.fraud
payment.saga.command.ledger
```



Participant service gửi reply qua Kafka:

```
payment.saga.reply.account
payment.saga.reply.fraud
payment.saga.reply.ledger
```



Hoặc có thể gom lại:

```
saga.commands
saga.replies
```



Nhưng trong production, thường tách topic theo domain hoặc theo loại message để dễ scale, phân quyền, monitor.

Một command nên có metadata đầy đủ:

```
{
  "eventId": "5b2f...",
  "sagaId": "a111...",
  "commandId": "c222...",
  "correlationId": "req-789",
  "causationId": "event-456",
  "businessKey": "PAYMENT-20260625-0001",
  "commandType": "ReserveMoney",
  "targetService": "account-service",
  "attempt": 1,
  "payload": {
    "accountId": "A123",
    "amount": "100.00",
    "currency": "USD"
  },
  "createdAt": "2026-06-25T10:00:00Z"
}
```

Reply cũng phải có sagaId , commandId , eventId :

```
{
  "eventId": "r333...",
  "sagaId": "a111...",
  "commandId": "c222...",
```

```
"replyType": "MoneyReserved",
"status": "SUCCESS",
"payload": {
  "reservationId": "R999"
},
"createdAt": "2026-06-25T10:00:02Z"
}
```

commandId rất quan trọng. Nó giúp participant service biết command này đã xử lý chưa. Nếu Kafka redeliver message, service không xử lý trùng.

5. Flow chi tiết từng bước của orchestrator

Giả sử API tạo payment.

Bước 1: API nhận request

Client gọi:

```
POST /payments
Idempotency-Key: abc-123
```

Payment Service không gọi ngay tất cả service. Nó tạo record payment và saga.

Trong một local DB transaction:

```
insert payment status = PENDING
insert saga_instance status = STARTED
insert outbox_event type = PaymentSagaStarted
commit
```



Pseudo code:

```
@Transactional
public PaymentResponse createPayment(CreatePaymentRequest request) {
    Payment payment = paymentRepository.save(
        Payment.pending(request.idempotencyKey(), request.amount())
    );

    SagaInstance saga = sagaRepository.save(
        SagaInstance.started("PaymentSaga", payment.getId())
    );
}
```

```

        outboxRepository.save(
            OutboxEvent.of(
                "PaymentSaga",
                saga.getId().toString(),
                "PaymentSagaStarted",
                Map.of("paymentId", payment.getId())
            )
        );

        return new PaymentResponse(payment.getId(), "PENDING");
    }

```

Tại sao không publish Kafka trực tiếp sau khi save DB?

Vì có dual-write problem.

Nếu làm:

save DB thành công
publish Kafka fail



Saga không bao giờ chạy.

Nếu làm:

publish Kafka thành công
save DB fail



Consumer nhận event nhưng DB không có payment.

Vì vậy dùng **Transactional Outbox Pattern**. Debezium mô tả outbox pattern là cách trao đổi dữ liệu an toàn giữa microservice, tránh inconsistency giữa internal database state và event mà service khác tiêu thụ. Debezium Outbox Event Router có thể capture thay đổi trong outbox table rồi route thành Kafka message.  Debezium

Bước 2: Outbox publisher đưa event lên Kafka

Có 2 cách phổ biến.

Cách 1: Debezium CDC.

Application ghi outbox_event vào DB
Debezium đọc transaction log DB
Debezium publish Kafka event



Ưu điểm:

DB commit rồi thì event chắc chắn được capture
Không cần app tự poll outbox
Giảm dual-write problem



Cách 2: App tự poll outbox.

Worker quét outbox_event status = NEW
publish Kafka
mark PUBLISHED



Cách này dễ triển khai hơn nhưng phải xử lý kỹ:

publish thành công nhưng update PUBLISHED fail
poller publish lại
consumer phải idempotent



Trong hệ thống lớn, Debezium CDC thường sạch hơn, nhưng vận hành Kafka Connect/Debezium cũng phức tạp hơn.

Bước 3: Orchestrator nhận PaymentSagaStarted

Orchestrator consume event PaymentSagaStarted .

Nó load saga từ DB, kiểm tra trạng thái hiện tại.

Saga STARTED
-> chuyển sang RESERVE_MONEY_PENDING
-> tạo command ReserveMoney
-> ghi command vào outbox



Tất cả xảy ra trong local DB transaction của orchestrator:

```
@Transactional
public void handlePaymentSagaStarted(Event event) {
    SagaInstance saga = sagaRepository.findByIdForUpdate(event.sagaId());

    if (!saga.isStarted()) {
        return; // duplicate event, ignore safely
    }

    UUID commandId = UUID.randomUUID();

    saga.moveTo("RESERVE_MONEY_PENDING");
}
```

```

sagaStepRepository.save(
    SagaStep.pending(saga.getId(), "RESERVE_MONEY", commandId)
);

outboxRepository.save(
    OutboxEvent.command(
        "account.commands",
        saga.getId(),
        commandId,
        "ReserveMoney",
        buildPayload(saga)
    )
);
}

```

Điểm quan trọng là orchestrator không chỉ "send Kafka message". Nó phải **lưu state trước hoặc cùng lúc với việc sinh command**.

Nếu orchestrator gửi command nhưng chưa lưu state, sau đó crash, khi nhận reply nó không biết mình đang ở bước nào.

Bước 4: Account Service xử lý command ReserveMoney

Account Service consume ReserveMoney .

Nó phải idempotent.

Trong DB transaction của Account Service:

```

check inbox_event xem commandId đã xử lý chưa
nếu đã xử lý -> không reserve lại
nếu chưa:
    insert inbox_event
    reserve money
    insert account_reservation
    insert outbox reply MoneyReserved / MoneyReserveFailed
commit

```



Pseudo:

```

@Transactional
public void handleReserveMoney(ReserveMoneyCommand command) {
    if (inboxRepository.exists(command.commandId())) {
        return;
    }
}

```

```

inboxRepository.save(command.commandId());

Account account = accountRepository.findByIdForUpdate(command.accountId());

if (account.availableBalance().compareTo(command.amount()) < 0) {
    outboxRepository.save(
        Reply.failed(command.sagaId(), command.commandId(), "INSUFFICIENT_FUNDS"
    );
    return;
}

Reservation reservation = account.reserve(command.amount());

reservationRepository.save(reservation);

outboxRepository.save(
    Reply.success(command.sagaId(), command.commandId(), "MoneyReserved",
        Map.of("reservationId", reservation.getId()))
);
}

```

Tại sao cần inbox table?

Vì Kafka thường được xử lý theo at-least-once trong nhiều hệ thống thực tế. Nếu consumer xử lý xong DB nhưng crash trước khi commit offset, sau khi restart nó có thể nhận lại message. Nếu không có inbox/dedup, nó reserve tiền lần nữa. Confluent docs cũng nhấn mạnh at-least-once có thể xử lý message nhiều hơn một lần; khi consumer xử lý xong nhưng crash trước khi lưu vị trí offset, message có thể được xử lý lại.

 Confluent Docum...

Bảng inbox:

```

CREATE TABLE inbox_event (
    event_id      UUID PRIMARY KEY,
    consumer_name VARCHAR(100) NOT NULL,
    processed_at  TIMESTAMP NOT NULL
);

```

Hoặc:

```

CREATE UNIQUE INDEX uk_inbox_consumer_command
ON inbox_event(consumer_name, event_id);

```


Bước 5: Orchestrator nhận reply MoneyReserved

Orchestrator consume reply:

MoneyReserved

- > load saga
- > check saga đang RESERVE_MONEY_PENDING không
- > mark step RESERVE_MONEY SUCCESS
- > chuyển sang FRAUD_CHECK_PENDING
- > sinh command CheckFraud



Nếu nhận duplicate reply:

replyEventId đã xử lý rồi
hoặc step đã SUCCESS rồi
=> ignore



Pseudo:

```
@Transactional
public void handleMoneyReserved(MoneyReservedReply reply) {
    if (inboxRepository.exists(reply.eventId())) {
        return;
    }

    inboxRepository.save(reply.eventId());

    SagaInstance saga = sagaRepository.findByIdForUpdate(reply.sagaId());

    if (!saga.isAt("RESERVE_MONEY_PENDING")) {
        return;
    }

    saga.markStepSuccess("RESERVE_MONEY");
    saga.moveTo("FRAUD_CHECK_PENDING");

    UUID commandId = UUID.randomUUID();

    sagaStepRepository.save(
        SagaStep.pending(saga.getId(), "CHECK_FRAUD", commandId)
    );

    outboxRepository.save(
        OutboxEvent.command(
            "fraud.commands",
            saga.getId(),
            commandId,
            "CheckFraud",

```

```
        buildFraudPayload(saga)
    )
};
}
```

Từ đây flow tiếp tục cho Fraud, Ledger, Payment finalization.

6. Compensation hoạt động thế nào?

Đây là phần nhiều người nói rất hời hợt.

Trong distributed transaction kiểu saga, khi một bước sau fail, bạn không rollback database transaction cũ theo nghĩa ACID. Vì transaction cũ đã commit ở service khác rồi.

Bạn phải tạo một transaction mới để "hoàn tác về mặt nghiệp vụ".

Ví dụ:

```
ReserveMoney thành công
FraudPassed thành công
CreateLedger fail
```



Orchestrator không thể rollback DB của Account Service. Nó gửi command:

```
ReleaseMoneyReservation
```



Account Service nhận command này và thực hiện local transaction:

```
reservation status RESERVED -> RELEASED
available balance tăng lại
publish MoneyReservationReleased
```



Camunda docs giải thích đúng tinh thần Saga: khi không thể rollback task đã làm, ta thực hiện hành động "undo" bằng compensation; BPMN compensation event có thể nối task chính với task undo tương ứng.  Camunda 8 Do...

Điểm cực kỳ quan trọng trong banking/finance:

```
Không phải compensation nào cũng là delete.
```



Ví dụ Ledger đã ghi bút toán thì không nên xóa dòng ledger. Cách đúng thường là ghi bút toán đảo:

Original ledger:

Debit A 100

Credit B 100

Compensation ledger:

Debit B 100

Credit A 100



Vì hệ thống tài chính cần audit trail. “Xóa cho sạch” là sai về mặt kế toán và audit.

Compensation phải idempotent:

Release reservation R999 nếu status đang RESERVED

Nếu đã RELEASED rồi thì return success



Pseudo:

```
@Transactional
public void releaseReservation(ReleaseReservationCommand command) {
    if (inboxRepository.exists(command.commandId())) {
        return;
    }

    inboxRepository.save(command.commandId());

    Reservation reservation = reservationRepository.findByIdForUpdate(
        command.reservationId()
    );

    if (reservation.isReleased()) {
        outboxRepository.save(Reply.success(...));
        return;
    }

    reservation.release();

    outboxRepository.save(Reply.success(...));
}
```

7. Retry: lỗi kỹ thuật khác lỗi nghiệp vụ

Orchestrator phải phân biệt rất rõ:

Technical failure



Business failure

Technical failure:

```
timeout
network error
database deadlock
Kafka temporary unavailable
service unavailable
```



Loại này có thể retry.

Business failure:

```
insufficient funds
fraud rejected
account closed
limit exceeded
invalid beneficiary
```



Loại này thường không retry vô nghĩa. Phải chuyển sang reject hoặc compensation.

Ví dụ:

```
ReserveMoney timeout
  -> retry 3 lần
  -> nếu vẫn không có reply, mark step TIMEOUT
  -> tùy business: compensation hoặc manual review

ReserveMoney trả INSUFFICIENT_FUNDS
  -> không retry
  -> mark saga FAILED
  -> update payment REJECTED
```



Retry policy nên có:

```
max attempts
backoff
jitter
timeout per step
dead-letter topic
manual recovery
```



Không nên retry vô hạn. Retry vô hạn có thể biến một lỗi nhỏ thành traffic storm.

8. Timeout trong orchestrator

Kafka không tự hiểu "bước này quá 5 phút chưa có reply thì fail". Orchestrator phải tự quản lý timeout.

Khi gửi command, saga step có `deadline_at` :

```
step = RESERVE_MONEY
status = PENDING
deadline_at = now + 30 seconds
```



Một scheduler quét:

```
SELECT *
FROM saga_step
WHERE status = 'PENDING'
AND deadline_at < now();
```

Nếu quá hạn:

```
retry nếu còn attempt
hoặc chuyển saga sang COMPENSATING
hoặc tạo incident/manual review
```



Scheduler có thể là:

```
Spring Scheduler
Quartz
Kubernetes CronJob
Dedicated timeout worker
Workflow engine timer như Camunda
```



Trong flow quan trọng, timeout không nên chỉ nằm trong memory. Nếu orchestrator pod chết, timeout vẫn phải tồn tại trong DB.

9. Kafka transaction có dùng được trong orchestrator không?

Có, nhưng phải hiểu đúng phạm vi.

Kafka transaction giúp trong case:

```
consume từ topic A
process
```



produce sang topic B
commit offset của topic A cùng transaction với output topic B

Nếu transaction abort, consumer đọc ở chế độ `read_committed` sẽ không thấy record bị abort. Kafka có `read_committed` và `read_uncommitted`; với `read_committed`, consumer chỉ đọc message từ transaction đã commit. [Confluent Docum...](#)

Spring Kafka hỗ trợ `KafkaTransactionManager`; nếu cấu hình `spring.kafka.producer.transaction-id-prefix`, Spring Boot có thể auto-configure `KafkaTransactionManager`, và `KafkaTemplate` trong transaction sẽ dùng transactional producer. Spring Kafka cũng lưu ý `transactionIdPrefix` phải unique giữa các application instance. [Home](#)

Nhưng điểm nguy hiểm là database + Kafka.

Spring Kafka có thể synchronize Kafka transaction với DB transaction, ví dụ DB commit trước rồi Kafka transaction commit sau. Nhưng tài liệu Spring cũng lưu ý nếu commit transaction thứ hai fail sau khi transaction chính đã commit, application phải có hành động remedial/compensation. [Home](#)

Nói thực tế:

Kafka transaction tốt cho Kafka-to-Kafka.
DB + Kafka thì outbox pattern thường an toàn và dễ kiểm soát hơn.

Vì vậy với orchestrator có DB state, mình thường chọn:

Local DB transaction
-> update saga state
-> insert outbox event
-> commit

Debezium / Outbox publisher
-> publish Kafka

Thay vì cố gắng buộc DB transaction và Kafka transaction thành một "distributed transaction" giả.

10. Ordering: làm sao giữ đúng thứ tự event?

Kafka chỉ đảm bảo ordering trong cùng một partition.

Vì vậy key của message rất quan trọng.

Với saga, thường dùng:

```
message key = sagaId
```



Hoặc với aggregate:

```
message key = paymentId  
message key = accountId
```



Nếu tất cả event của cùng một saga có cùng key, chúng sẽ vào cùng partition, consumer trong cùng group xử lý theo thứ tự partition.

Ví dụ:

```
sagaId A -> partition 1  
sagaId B -> partition 2  
sagaId C -> partition 1
```



Điều này giúp tránh trường hợp orchestrator nhận LedgerCreated trước MoneyReserved cho cùng một saga.

Nhưng không nên quá tin vào ordering để thay thế state validation. Orchestrator vẫn phải check:

```
Tôi đang chờ reply step nào?  
Reply này có đúng commandId không?  
Reply này có bị duplicate không?  
State hiện tại có cho phép transition này không?
```



Nếu nhận event không hợp lệ, không nên update state bừa. Có thể log, ignore, hoặc đưa vào incident.

11. Concurrency: nhiều orchestrator pod chạy thì có bị đụng nhau không?

Trong Kubernetes, orchestrator thường chạy nhiều pod:

```
orchestrator pod 1  
orchestrator pod 2  
orchestrator pod 3
```



Nếu cùng một saga event bị xử lý bởi 2 pod thì sao?

Có vài lớp bảo vệ.

Lớp 1: Kafka consumer group

Trong cùng consumer group, một partition tại một thời điểm thường chỉ được assign cho một consumer instance. Nếu key theo `sagaId`, event cùng saga đi cùng partition, khả năng cùng lúc xử lý song song cùng saga giảm đi.

Nhưng vẫn có case crash/rebalance/redelivery. Không được dựa hoàn toàn vào Kafka assignment.

Lớp 2: DB lock hoặc optimistic locking

Khi update saga, dùng:

```
SELECT * FROM saga_instance
WHERE id = ?
FOR UPDATE;
```

Hoặc dùng optimistic locking:

```
version = 5
update where id = ? and version = 5
```



Nếu update count = 0, nghĩa là có pod khác đã update trước.

Ví dụ:

```
UPDATE saga_instance
SET status = 'FRAUD_CHECK_PENDING',
    version = version + 1
WHERE id = :sagaId
AND version = :oldVersion;
```

Lớp 3: idempotency/inbox

Mỗi event/reply có `eventId`. Nếu event đã xử lý rồi thì bỏ qua.

```
CREATE UNIQUE INDEX uk_saga_inbox
ON saga_inbox(event_id, consumer_name);
```

Nhờ vậy, dù Kafka deliver lại message, orchestrator không chạy state transition hai lần.

12. Outbox + Inbox là cặp bắt buộc trong thiết kế nghiêm túc

Nhiều người chỉ biết outbox nhưng quên inbox.

Outbox giải quyết chiều publish

Service update DB và muốn publish event.

DB update thành công nhưng Kafka publish fail



Outbox giải quyết bằng cách ghi event vào DB cùng transaction với business data.

Inbox giải quyết chiều consume

Service nhận message và update DB.

DB update thành công nhưng consumer crash trước khi commit offset
Kafka deliver lại message



Inbox giải quyết bằng cách lưu eventId/commandId đã xử lý.

Production flow đầy đủ:

Consumer nhận message

- > begin DB transaction
- > check inbox
- > nếu đã xử lý: commit DB, commit offset
- > nếu chưa:
 - insert inbox
 - update business data
 - insert outbox reply/event
- commit DB
- > commit Kafka offset



Nếu crash sau commit DB nhưng trước commit offset:

Kafka gửi lại message
Inbox thấy đã xử lý
Không làm lại business side effect
Commit offset



Đó là cách bạn đạt “effectively once” ở tầng business, dù messaging thực tế có thể at-least-once.

13. Thiết kế topic cho orchestrator

Một thiết kế đơn giản:

```
payment.saga.started  
account.command  
account.reply  
fraud.command  
fraud.reply  
ledger.command  
ledger.reply  
payment.saga.completed  
payment.saga.failed
```



Một thiết kế generic hơn:

```
saga.commands  
saga.replies  
saga.events
```



Trong enterprise, mình thường thích tách vừa đủ:

```
payment-orchestrator.commands.account  
payment-orchestrator.commands.fraud  
payment-orchestrator.commands.ledger  
  
payment-orchestrator.replies.account  
payment-orchestrator.replies.fraud  
payment-orchestrator.replies.ledger
```



Lý do:

```
Dễ phân quyền ACL  
Dễ monitor lag từng service  
Dễ scale consumer riêng  
Dễ DLQ riêng  
Dễ debug
```



Mỗi topic nên có DLQ:

```
account.command.dlq  
fraud.command.dlq  
ledger.command.dlq
```



Nhưng DLQ không phải thùng rác. Phải có quy trình xử lý DLQ:

xem lỗi
sửa data/code
replay message
hoặc mark incident/manual failed



14. State machine của Payment Saga

Ví dụ state machine:

```

STARTED
|
v
RESERVE_MONEY_PENDING
| success
v
FRAUD_CHECK_PENDING
| success
v
LEDGER_CREATE_PENDING
| success
v
PAYMENT_COMPLETE_PENDING
| success
v
COMPLETED

```



Nếu fraud reject:

```

FRAUD_CHECK_PENDING
| rejected
v
RELEASE_MONEY_PENDING
| success
v
FAILED

```



Nếu ledger fail sau khi money reserved:

```

LEDGER_CREATE_PENDING
| failed/timeout
v
COMPENSATING
|
v
RELEASE_MONEY_PENDING
|

```



V
COMPENSATED_FAILED

Một transition hợp lệ phải được kiểm soát rõ.

Ví dụ không cho phép:

STARTED -> LEDGER_CREATE_PENDING
COMPLETED -> RESERVE_MONEY_PENDING
FAILED -> FRAUD_CHECK_PENDING



Nếu không kiểm soát state machine, duplicate/out-of-order event có thể làm saga nhảy sai trạng thái.

15. Cách triển khai orchestrator bằng Java/Spring Boot

15.1 Các module chính

Một orchestrator service thường có:

API layer
Kafka consumers
Saga state machine
Saga repository
Outbox repository
Inbox repository
Timeout scheduler
Retry handler
Compensation handler
Observability



Nhưng thay vì nghĩ theo package, hãy nghĩ theo trách nhiệm.

SagaStateMachine

Nơi quyết định:

Event này đến thì state chuyển sang đâu?
Có sinh command mới không?
Có cần compensation không?
Có cần final event không?



Ví dụ:

```
public SagaDecision onMoneyReserved(SagaInstance saga, MoneyReservedReply reply) {
    if (!saga.isAt("RESERVE_MONEY_PENDING")) {
        return SagaDecision.ignore();
    }

    saga.moveTo("FRAUD_CHECK_PENDING");

    return SagaDecision.sendCommand(
        "fraud.commands",
        new CheckFraudCommand(saga.getId(), saga.getBusinessKey())
    );
}
```

SagaRepository

Lưu state vào DB.

OutboxRepository

Lưu command/event cần publish.

InboxRepository

Dedup message đã consume.

15.2 Consumer handler mẫu

```
@KafkaListener(topics = "account.reply", groupId = "payment-orchestrator")
public void onAccountReply(AccountReply reply) {
    sagaApplicationService.handleAccountReply(reply);
}
```

Service:

```
@Transactional
public void handleAccountReply(AccountReply reply) {
    if (inboxRepository.exists(reply.eventId(), "payment-orchestrator")) {
        return;
    }

    inboxRepository.save(reply.eventId(), "payment-orchestrator");
}
```

```

SagaInstance saga = sagaRepository.findByIdForUpdate(reply.sagaId());

SagaDecision decision = stateMachine.handle(saga, reply);

sagaRepository.save(saga);

decision.commands().forEach(command ->
    outboxRepository.save(command.toOutboxEvent())
);
}

```

Điểm quan trọng:

Consume message
Dedup
Lock saga
Apply state transition
Write outbox
Commit DB



Không publish Kafka trực tiếp trong state machine transaction nếu bạn đang dùng outbox.

15.3 Participant service mẫu

Ví dụ Account Service.

```

@KafkaListener(topics = "account.command", groupId = "account-service")
public void onReserveMoney(ReserveMoneyCommand command) {
    accountCommandService.reserveMoney(command);
}

```

Service:

```

@Transactional
public void reserveMoney(ReserveMoneyCommand command) {
    if (inboxRepository.exists(command.commandId(), "account-service")) {
        return;
    }

    inboxRepository.save(command.commandId(), "account-service");

    Account account = accountRepository.findByIdForUpdate(command.accountId());
}

```

```
if (!account.canReserve(command.amount())) {
    outboxRepository.save(
        ReplyEvent.failed(
            command.sagaId(),
            command.commandId(),
            "MoneyReserveFailed",
            "INSUFFICIENT_FUNDS"
        )
    );
    return;
}

Reservation reservation = account.reserve(command.amount());

reservationRepository.save(reservation);

outboxRepository.save(
    ReplyEvent.success(
        command.sagaId(),
        command.commandId(),
        "MoneyReserved",
        Map.of("reservationId", reservation.getId())
    )
);
}
```

16. Orchestrator nên dùng custom code hay workflow engine?

Có 2 hướng.

Hướng 1: Tự viết orchestrator bằng Java/Spring Boot

Phù hợp khi:

Flow không quá phức tạp
Team muốn kiểm soát code hoàn toàn
Throughput cao
Ít cần UI nghiệp vụ
Ít cần BPMN/manual task



Bạn tự quản lý:

saga table
state machine
retry
timeout



compensation
outbox/inbox
monitoring

Ưu điểm:

nhẹ
nhanh
fit với domain
ít dependency lớn



Nhược điểm:

phải tự làm nhiều thứ
debug workflow khó nếu không có UI
business thay đổi flow phải sửa code
manual intervention phải tự xây



Hướng 2: Dùng workflow engine như Camunda

Phù hợp khi:

Flow nhiều bước
Có human approval
Có timeout phức tạp
Có manual task
Có BPMN diagram
Cần audit workflow rõ
Business muốn nhìn process
Compensation phức tạp



Camunda 8 có tài liệu hướng dẫn orchestration microservices, và docs của Camunda cũng mô tả cách dùng Saga/BPMN compensation để xử lý long-running transaction khi không thể rollback task đã làm. 📖 Camunda 8 Docu... +1

Ưu điểm:

workflow nhìn được bằng diagram
có timer/retry/incident
audit tốt
dễ quản lý process dài



Nhược điểm:

thêm hạ tầng
thêm learning curve



phải vận hành workflow engine
cần design worker/idempotency cẩn thận

Điểm rất quan trọng: dù dùng Camunda hay custom orchestrator, participant service vẫn phải idempotent. Workflow engine không tự làm DB side effect của bạn trở nên exactly-once.

17. Các lỗi production hay gặp với orchestrator

Lỗi 1: Orchestrator lưu state trong memory

Nếu pod chết, saga mất state.

Sai:

Map<sagaId, state> in memory



Đúng:

Saga state lưu DB
Pod stateless
Restart vẫn resume được



Lỗi 2: Không có idempotency ở participant

Kafka redeliver command, participant xử lý lại.

Ví dụ:

ReserveMoney command xử lý 2 lần



Cách giải quyết:

commandId unique
inbox table
business unique constraint
idempotent operation



Lỗi 3: Compensation không idempotent

Orchestrator gửi ReleaseMoney 2 lần. Nếu Account Service cộng lại available balance 2 lần thì sai nghiêm trọng.

Đúng là release phải dựa trên trạng thái reservation:

```
if reservation.status == RELEASED:
    return success
else:
    release once
```



Lỗi 4: Không phân biệt business error và technical error

Nếu insufficient funds mà retry 10 lần thì vô nghĩa.

Nếu timeout mà mark payment failed ngay cũng có thể sai, vì service downstream có thể đã xử lý thành công nhưng reply bị chậm.

Cần policy rõ:

```
Business rejection -> fail/compensate
Technical error -> retry/backoff/timeout/manual review
Unknown result -> query status hoặc manual reconciliation
```



Lỗi 5: Không có query status

Orchestrator gửi command, service xử lý xong nhưng reply mất.

Nếu chỉ chờ reply, saga timeout và compensation sai.

Giải pháp cho nghiệp vụ quan trọng:

```
command có idempotency key
participant lưu command/result
orchestrator có thể gửi QueryCommandStatus
hoặc participant reply lại result cũ nếu nhận duplicate command
```



Ví dụ Account Service nhận lại ReserveMoney commandId = C1 :

```
Nếu C1 đã xử lý thành công:
    gửi lại MoneyReserved với reservationId cũ
```



Đây là pattern rất hữu ích.

Lỗi 6: Dùng Redis lock thay cho saga state

Redis lock không đủ để quản lý business transaction dài.

Saga có thể kéo dài vài giây, vài phút, vài giờ. Redis lock có TTL, failover, eviction. Không nên dùng Redis làm nơi chính để nhớ saga đang ở đâu.

Redis có thể dùng phụ trợ:

```
rate limit
short-lived lock giảm duplicate
cache saga read model
```



Nhưng saga state chính nên nằm trong durable DB/workflow engine.

Lỗi 7: Không có observability

Khi production lỗi, bạn cần trả lời:

```
Payment này đang ở step nào?
Command cuối gửi lúc nào?
Service nào chưa reply?
Đã retry mấy lần?
Có compensation chưa?
EventId là gì?
Kafka offset nào?
CorrelationId là gì?
```



Nếu không có các thông tin này, debug saga rất cực.

Nên log theo:

```
sagaId
businessKey
correlationId
commandId
eventId
stepName
stateBefore
stateAfter
attempt
topic
partition
offset
```



18. Orchestrator triển khai trên Kubernetes như thế nào?

Orchestrator service nên stateless:

Nhiều pod orchestrator
State nằm trong DB
Message nằm trong Kafka
Outbox/inbox nằm trong DB



Deployment:

payment-orchestrator pod 1
payment-orchestrator pod 2
payment-orchestrator pod 3



Scale theo:

Kafka consumer lag
CPU
DB latency
number of pending saga steps
timeout backlog



Nhưng phải nhớ: scale orchestrator pod không làm một saga chạy song song hơn nếu tất cả event cùng key vào một partition. Muốn tăng throughput tổng thể, cần:

nhiều sagaId khác nhau
topic đủ partitions
consumer instances phù hợp
DB đủ capacity



Nếu topic chỉ có 3 partitions, consumer group tối đa chỉ có 3 consumer active cho topic đó. Thêm 10 pod orchestrator không giúp tăng consume parallelism cho topic đó.

19. Một thiết kế hoàn chỉnh mẫu

Thành phần

payment-service
account-service
fraud-service
ledger-service
payment-orchestrator



Kafka
PostgreSQL/MySQL
Debezium/Kafka Connect hoặc Outbox Poller
Prometheus/Grafana
DLQ tooling

Database mỗi service

Payment DB:

payment
saga_instance
saga_step
outbox_event
inbox_event



Account DB:

account
reservation
outbox_event
inbox_event



Ledger DB:

ledger_entry
outbox_event
inbox_event



Topic

payment.saga.started
account.commands
account.replies
fraud.commands
fraud.replies
ledger.commands
ledger.replies
payment.saga.completed
payment.saga.failed



Flow thành công

1. Client POST /payments
2. Payment Service tạo payment PENDING + saga STARTED + outbox PaymentSagaStart
3. Outbox publish PaymentSagaStarted
4. Orchestrator nhận event



5. Orchestrator ghi state RESERVE_MONEY_PENDING + outbox ReserveMoney
6. Account Service nhận ReserveMoney
7. Account Service reserve tiền + outbox MoneyReserved
8. Orchestrator nhận MoneyReserved
9. Orchestrator chuyển sang FRAUD_CHECK_PENDING + outbox CheckFraud
10. Fraud Service reply FraudPassed
11. Orchestrator chuyển sang LEDGER_PENDING + outbox CreateLedger
12. Ledger Service tạo ledger + reply LedgerCreated
13. Orchestrator mark saga COMPLETED + update payment SUCCESS
14. Publish PaymentCompleted

Flow thất bại

1. Money reserved thành công
2. Fraud reject
3. Orchestrator chuyển COMPENSATING
4. Gửi ReleaseMoney
5. Account Service release reservation
6. Orchestrator mark payment FAILED
7. Publish PaymentFailed



20. Câu trả lời senior nên nói trong phỏng vấn

Nếu bị hỏi:

Làm sao quản lý transaction phân tán giữa các service dùng Kafka?



Bạn có thể trả lời như này:

Em sẽ không cố dùng một distributed ACID transaction giữa nhiều microservice, \

Nếu flow đơn giản, có thể dùng choreography bằng domain events. Nhưng với nghiê

Kafka sẽ đóng vai trò message backbone để orchestrator gửi command và nhận rep

Em không tin Kafka exactly-once là đủ cho toàn bộ business side effect, vì exa

Orchestrator sẽ có timeout scheduler, retry policy phân biệt technical error và



Đây là câu trả lời khá mạnh cho senior.

21. Checklist triển khai orchestrator thực tế

Khi thiết kế thật, bạn đi theo thứ tự này:

1. Xác định boundary của saga.
2. Xác định service nào sở hữu dữ liệu nào.
3. Vẽ state machine.
4. Xác định từng command và reply.
5. Thiết kế compensation cho từng step đã commit.
6. Thiết kế saga_instance và saga_step table.
7. Thiết kế outbox/inbox.
8. Thiết kế Kafka topic và message key.
9. Thêm idempotency key, commandId, eventId, correlationId.
10. Thêm retry policy cho technical error.
11. Thêm business failure handling.
12. Thêm timeout scanner.
13. Thêm DLQ và replay tool.
14. Thêm monitoring theo saga state.
15. Test crash ở mọi điểm nguy hiểm.



Test failure matrix nên có:

Crash sau khi update DB nhưng trước khi ghi outbox
 Crash sau khi ghi outbox nhưng trước khi publish Kafka
 Publish Kafka thành công nhưng update outbox status fail
 Consumer xử lý DB xong nhưng chưa commit offset
 Reply duplicate
 Reply đến trễ
 Reply đến sai state
 Participant timeout nhưng sau đó vẫn reply success
 Compensation command duplicate
 Orchestrator pod restart giữa saga
 Kafka rebalance khi đang xử lý
 DB deadlock
 DLQ replay



Nếu bạn test được các case này, orchestrator của bạn mới đáng tin.

22. Kết luận gốc rễ

Toàn bộ vấn đề này quy về một nguyên tắc:

Trong microservice, không cố ép nhiều service thành một transaction ACID khổng lồ
 Thay vào đó, chia thành nhiều local transaction nhỏ,
 liên kết bằng message,
 quản lý bằng saga state,
 và sửa sai bằng compensation + idempotency + reconciliation.



Kafka giúp vận chuyển event/command bền vững.

Orchestrator giúp biết business process đang ở đâu.

Outbox giúp không mất event sau DB commit.

Inbox giúp không xử lý trùng khi message bị redeliver.

Compensation giúp hoàn tác nghiệp vụ khi không thể rollback kỹ thuật.

Reconciliation giúp phát hiện và sửa những trường hợp hiếm vẩn lệch.

Câu chốt rất quan trọng:

Distributed transaction trong microservice không phải là “rollback tất cả”.
Nó là “thiết kế để hệ thống luôn đi đến một trạng thái cuối cùng đúng, kể cả khi message duplicate, service crash, network timeout, hoặc Kafka redeli



... Nguồn

giải thích chi tiết về lazy, eager trong java. lý do vì sao lazy load gây ra n+1 query, cách giải quyết. mình không cần lý thuyết suông, hãy nói về thực tế, đi sâu vào vấn đề luôn.

Mình sẽ giải thích theo kiểu bạn đang debug một service Spring Boot thật: entity mapping, câu SQL thực sự bắn ra, vì sao N+1 xuất hiện, và khi nào dùng JOIN FETCH , EntityGraph , batch fetch, projection, DTO, hoặc tách query.

Đã suy nghĩ trong 37s >

Để nói đúng thực tế: **lazy/eager không phải là “load chậm/load sớm” đơn giản**. Trong Java backend dùng Spring Boot + JPA/Hibernate, nó là câu hỏi:

Khi query entity chính, có nên kéo luôn object liên quan không?
Nếu không kéo luôn, lúc nào Hibernate sẽ bắn SQL bổ sung?
Nếu đang trả API list 100 record, có vô tình bắn thêm 100 query không?



Đây là chỗ gây ra **N+1 query**, performance tụt rất mạnh trong production.

1. Lazy và Eager trong thực tế là gì?

Giả sử có hệ thống banking:

```
@Entity
public class Account {
```



```
@Id
private Long id;

private String accountNumber;

@ManyToOne(fetch = FetchType.LAZY)
private Customer customer;

@OneToMany(mappedBy = "account", fetch = FetchType.LAZY)
private List<Transaction> transactions;
}
```

Account là tài khoản.

Customer là chủ tài khoản.

Transaction là danh sách giao dịch.

Khi bạn gọi:

```
Account account = accountRepository.findById(1L).orElseThrow();
```

Nếu customer là **LAZY**, Hibernate chỉ load bằng account trước:

```
select *
from account
where id = 1;
```

Nó **chưa load customer ngay**. Trong object account, field customer chưa phải object thật đầy đủ, mà thường là một proxy. Đến khi code dùng vào:

```
account.getCustomer().getName();
```

Hibernate mới bắn thêm query:

```
select *
from customer
where id = ?;
```

Đó là lazy loading: **load khi thật sự cần dùng**. Hibernate docs cũng mô tả eager là fetch ngay cùng lúc với owner, còn lazy là fetch khi value được access; tài liệu Hibernate còn

tách rõ hai câu hỏi: "khi nào fetch?" và "fetch bằng cách nào?"  Hibernate Docu...

2. Lazy không xấu. Vấn đề là lazy bị dùng sai ngữ cảnh

Lazy rất tốt khi bạn load một entity nhưng không phải lúc nào cũng cần dữ liệu liên quan.

Ví dụ API:

```
GET /accounts/1
```

Response chỉ cần:

```
{
  "id": 1,
  "accountNumber": "123456",
  "status": "ACTIVE"
}
```

Nếu lúc nào cũng kéo `customer`, `transactions`, `cards`, `limits`, `branch`, thì query sẽ rất nặng.

Lazy giúp tránh chuyện đó.

Nhưng vấn đề xuất hiện khi bạn query **một danh sách entity**, sau đó trong vòng lặp lại access association lazy.

Ví dụ:

```
List<Account> accounts = accountRepository.findAll();

for (Account account : accounts) {
    String customerName = account.getCustomer().getName();
}
```

Đây là nguồn gốc rất phổ biến của N+1.

3. N+1 query là gì? Đi từng bước thật chậm

Giả sử DB có 100 account.

Code:

```
List<Account> accounts = accountRepository.findAll();

for (Account account : accounts) {
    System.out.println(account.getCustomer().getName());
}
```

Query đầu tiên:

```
select *
from account;
```

Đây là query số 1. Nó lấy ra 100 account.

Nhưng vì customer là lazy, Hibernate chưa load customer.

Sau đó vòng lặp chạy.

Account thứ nhất:

```
account1.getCustomer().getName();
```

Hibernate bắt:

```
select *
from customer
where id = 10;
```

Account thứ hai:

```
account2.getCustomer().getName();
```

Hibernate bắt tiếp:

```
select *  
from customer  
where id = 11;
```

Cứ như vậy cho 100 account.

Tổng cộng:

```
1 query lấy account  
+ 100 query lấy customer  
= 101 queries
```



Đó là **N+1 query**.

```
N = số record chính  
+1 = query ban đầu lấy danh sách chính
```



Nếu API trả 20 account, có thể là 21 queries.

Nếu API trả 500 account, có thể là 501 queries.

Nếu bên dưới mỗi account lại có transactions, cards, limits thì số query còn kinh khủng hơn.

Hibernate docs cũng nói fetch kiểu `SELECT` dùng separate SQL select để load dữ liệu, có thể eager hoặc lazy, và đây là strategy thường được gọi là N+1; tài liệu cũng cảnh báo nếu quên `JOIN FETCH` các eager association trong JPQL, Hibernate có thể bắn secondary select cho từng association và dẫn đến N+1.  [Hibernate Docum...](#)

4. Tại sao lazy gây ra N+1?

Nói chính xác hơn: **lazy không tự gây ra N+1**. Lazy chỉ trì hoãn query. N+1 xảy ra khi bạn trì hoãn query rồi lại access association đó nhiều lần trong vòng lặp.

Lazy giống như bạn nói với Hibernate:

```
Tạm thời đừng load customer.  
Khi nào tôi gọi getCustomer() thì hãy load.
```



Sau đó bạn lại làm:

```
for (Account account : accounts) {  
    account.getCustomer().getName();  
}
```

Tức là bạn đang bảo Hibernate:

```
Load customer cho account 1.  
Load customer cho account 2.  
Load customer cho account 3.  
...  
Load customer cho account 100.
```



Hibernate không tự biết ngay từ đầu rằng bạn sẽ cần customer của toàn bộ 100 account, trừ khi bạn nói rõ trong query.

Vấn đề gốc là: **fetch plan không khớp với use case.**

Bạn query list account, nhưng use case thật sự lại cần account + customer. Nếu query ban đầu chỉ lấy account, Hibernate phải bổ sung từng query sau đó.

5. Eager có giải quyết N+1 không?

Nhiều người nghĩ:

```
@ManyToOne(fetch = FetchType.EAGER)  
private Customer customer;
```

là sẽ hết N+1.

Không chắc.

Eager nghĩa là association này **bắt buộc phải được load** khi load entity owner. Nhưng cách load có thể không phải lúc nào cũng là một SQL join đẹp.

Ví dụ với `entityManager.find(Account.class, id)`, Hibernate có thể join luôn customer. Nhưng với JPQL như:

```
@Query("select a from Account a")  
List<Account> findAllAccounts();
```

Nếu query không có `JOIN FETCH`, Hibernate vẫn có thể query account trước, rồi bắt thêm select để load eager association. Hibernate docs cảnh báo rất rõ: eager association bị omit trong JPQL có thể được initialize bằng secondary statement và gây N+1; tài liệu cũng nói eager fetching không thể override per query nên association luôn bị retrieve kể cả khi bạn không cần, vì vậy nên tránh eager và để association lazy mặc định.  [Hibernate Docum...](#)

Nói thực tế:

EAGER không phải thuộc trị N+1. 
EAGER nhiều khi còn làm hệ thống tệ hơn vì lúc nào cũng load dữ liệu không cần.

Ví dụ API list account chỉ cần:

```
{
  "id": 1,
  "accountNumber": "123456"
}
```

Nhưng vì bạn set customer eager, Hibernate vẫn phải load customer. Sau này customer lại eager sang address, address eager sang city, city eager sang country. Cuối cùng query rất nặng mà API không cần.

Đó là lý do trong enterprise, convention thường là:

```
@ManyToOne(fetch = FetchType.LAZY)
@OneToOne(fetch = FetchType.LAZY)
@OneToMany(fetch = FetchType.LAZY)
@ManyToMany(fetch = FetchType.LAZY)
```

Hibernate cũng khuyến nghị đánh dấu static association là lazy và dùng dynamic fetching strategy cho từng use case cụ thể.  [Hibernate Docum...](#)

6. Ví dụ production: API list account bị chậm vì N+1

Giả sử bạn có API:

```
GET /accounts?page=0&size=50
```

Response cần:

```
[
  {
    "accountId": 1,
    "accountNumber": "123",
    "customerName": "Nguyen Van A",
    "balance": 1000
  }
]
```

Code ban đầu:

```
@Transactional(readOnly = true)
public List<AccountResponse> getAccounts() {
    List<Account> accounts = accountRepository.findAll();

    return accounts.stream()
        .map(account -> new AccountResponse(
            account.getId(),
            account.getAccountNumber(),
            account.getCustomer().getName(),
            account.getBalance()
        ))
        .toList();
}
```

Nhìn rất bình thường. Nhưng thực tế SQL:

```
select * from account;
```

Sau đó 50 lần:

```
select * from customer where id = ?;
```

Tổng 51 query.

Nếu customer lại có branch lazy và response cần branch name:

```
account.getCustomer() \ getBranch() \ getName()
// ...
```

Có thể thành:

```
1 query account
+ 50 query customer
+ 50 query branch
= 101 queries
```

Nếu production mỗi giây có 100 request, bạn đã biến 100 request thành 10,100 SQL queries.

Đây là lý do N+1 nguy hiểm: local test data ít thì không thấy. Production data nhiều mới thấy DB CPU tăng, connection pool full, API latency tăng.

7. Cách giải quyết số 1: JOIN FETCH

Nếu use case cần account + customer, hãy nói rõ trong query:

```
@Query("""
    select a
    from Account a
    join fetch a.customer
    where a.status = :status
""")
List<Account> findByStatusWithCustomer(AccountStatus status);
```

SQL sẽ gần như:

```
select a.*, c.*
from account a
join customer c on a.customer_id = c.id
where a.status = ?;
```

Lúc này thay vì:

```
1 query account
+ N query customer
```

nó thành:

1 query account join customer



Đây là cách rất hiệu quả cho ManyToOne hoặc OneToOne .

Hibernate docs nói để đạt performance tốt với HQL, bạn sẽ cần dùng join fetch khá thường xuyên; nếu không, rất dễ gặp "n+1 selects". Tài liệu cũng minh họa join fetch dùng để fetch association trong cùng một SQL query. [Hibernate Docum...](#)

Nhưng JOIN FETCH không phải lúc nào cũng dùng bừa

Với ManyToOne :

Account -> Customer

JOIN FETCH thường ổn vì mỗi account có một customer.

Nhưng với OneToMany :

Customer -> List<Account>

Cẩn thận hơn.

Ví dụ:

```
@Query("""
    select c
    from Customer c
    join fetch c.accounts
""")
List<Customer> findAllWithAccounts();
```

Nếu 1 customer có 10 account, SQL result sẽ nhận dòng:

Customer A + Account 1
Customer A + Account 2
Customer A + Account 3
...



Ở Java, Hibernate có thể gom lại thành một Customer với list accounts, nhưng SQL result set vẫn bị phình.

Nếu fetch nhiều collection cùng lúc:

```
join fetch c.accounts  
join fetch c.cards  
join fetch c.addresses
```

Có thể gây **cartesian product**.

Ví dụ:

```
1 customer có 10 accounts  
1 customer có 5 cards  
1 customer có 3 addresses
```



SQL có thể ra:

```
10 x 5 x 3 = 150 rows cho 1 customer
```



Cho nên JOIN FETCH tốt, nhưng phải dùng đúng.

8. Cách giải quyết số 2: DTO projection — thực tế nhiều API nên dùng cách này

Nếu API chỉ cần vài field, đừng load nguyên entity graph.

Ví dụ response cần:

```
{  
  "accountId": 1,  
  "accountNumber": "123",  
  "customerName": "Nguyen Van A"  
}
```

Bạn có thể query thẳng DTO:

```
@Query("""
    select new com.example.AccountSummaryResponse(
        a.id,
        a.accountNumber,
        c.name
    )
    from Account a
    join a.customer c
    where a.status = :status
""")
List<AccountSummaryResponse> findAccountSummaries(AccountStatus status);
```

Khi đó Hibernate không cần tạo entity `Customer` đầy đủ. Nó chỉ lấy đúng field cần trả API.

SQL:

```
select a.id, a.account_number, c.name
from account a
join customer c on a.customer_id = c.id
where a.status = ?;
```

Đây là cách rất hay dùng trong enterprise cho API list/search/report.

Tư duy thực tế:

Nếu API chỉ đọc dữ liệu để hiển thị, ưu tiên DTO projection.
Nếu use case cần modify entity, dùng entity.



Ví dụ dashboard, search page, listing page, export report: DTO projection thường tốt hơn entity graph.

9. Cách giải quyết số 3: EntityGraph

Nếu bạn không muốn viết JPQL `JOIN FETCH` cho từng method, có thể dùng `@EntityGraph`.

Ví dụ:

```
@EntityGraph(attributePaths = {"customer"})
List<Account> findByStatus(AccountStatus status);
```

Spring Data JPA `@EntityGraph` cho phép cấu hình entity graph trên repository method; `attributePaths` có thể chỉ định direct property hoặc nested property như `property.nestedProperty`.  Home

Ví dụ nested:

```
@EntityGraph(attributePaths = {"customer", "customer.branch"})
List<Account> findByStatus(AccountStatus status);
```

Dùng tốt khi:

Cùng một entity nhưng mỗi API cần fetch graph khác nhau.
Không muốn đổi mapping từ LAZY sang EAGER.
Muốn giữ repository method dễ đọc.



Nhưng EntityGraph cũng không phải magic. Nếu fetch nhiều collection lớn, vẫn có thể làm query phình. Nó giải quyết fetch plan, không giải quyết vấn đề bạn đang cố kéo quá nhiều data.

10. Cách giải quyết số 4: BatchSize — giảm N+1 chứ không phải luôn xóa N+1

Giả sử bạn có 100 account, mỗi account lazy customer.

Nếu access từng customer, bình thường có thể là:

1 query account
+ 100 query customer



Nếu bật batch fetching, Hibernate có thể gom nhiều id lại:

```
select *
from customer
where id in (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
```

Ví dụ batch size 10:

1 query account
+ 10 query customer batches
= 11 queries



Mapping:

```
@ManyToOne(fetch = FetchType.LAZY)
@BatchSize(size = 50)
private Customer customer;
```

Hoặc collection:

```
@OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
@BatchSize(size = 50)
private List<Account> accounts;
```

Hibernate docs mô tả @BatchSize dùng khi fetch uninitialized proxies và cho phép load nhiều entity/collection trong một database round trip; tài liệu cũng nói không có @BatchSize thì sẽ gặp N+1, nhưng đa số trường hợp DTO projection hoặc JOIN FETCH vẫn là lựa chọn tốt hơn nếu cần lấy toàn bộ data bằng một query. [Hibernate Docum...](#)

Nói thực tế:

JOIN FETCH: tôi biết chắc use case này cần data liên quan ngay.
BatchSize: tôi không chắc lúc nào cần, nhưng nếu cần nhiều cái thì gom query lại.
DTO projection: tôi chỉ cần field để trả response, không cần entity đầy đủ.



11. Cách giải quyết số 5: FetchType.SUBSELECT cho collection

Với OneToMany, nếu bạn load nhiều parent rồi access collection của từng parent, Hibernate có thể dùng subselect để tránh mỗi parent một query.

Ví dụ:

```
@OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
@Fetch(FetchMode.SUBSELECT)
private List<Employee> employees;
```

Flow:

```
List<Department> departments = departmentRepository.findByNameLike("IT%");

for (Department d : departments) {
    d.getEmployees().size();
}
```

Thay vì:

1 query department
+ N query employees theo từng department



Hibernate có thể dùng một query để load employees cho toàn bộ department vừa fetch. Hibernate docs nói `FetchMode.SUBSELECT` giúp tránh N+1 bằng cách generate một SQL statement để initialize collection cho tất cả parent entities đã được fetch trước đó.

 Hibernate Docum...

Nhưng dùng cái này cũng phải cẩn thận. Nếu query parent ban đầu lấy quá nhiều record, subselect có thể kéo rất nhiều child.

12. Vấn đề lớn: JOIN FETCH với pagination

Đây là lỗi cực hay gặp.

Ví dụ:

```
@Query("""
    select c
    from Customer c
    join fetch c.accounts
""")
Page<Customer> findCustomers(Pageable pageable);
```

Bạn nghĩ DB sẽ lấy 20 customer và account của 20 customer.

Nhưng với `OneToMany`, SQL join làm số dòng bị nhân lên. Pagination trên joined rows có thể sai về mặt logic.

Ví dụ customer A có 100 accounts. Nếu DB limit 20 rows, có thể bạn chỉ lấy được 20 dòng đầu của customer A, không phải 20 customer.

Cách xử lý thực tế thường là **two-step query**.

Bước 1: query page id của parent.

```
@Query("""
    select c.id
    from Customer c
    where c.status = :status
    order by c.createdAt desc
""")
Page<Long> findCustomerIds(AccountStatus status, Pageable pageable);
```

Bước 2: fetch data theo ids:

```
@Query("""
    select distinct c
    from Customer c
    left join fetch c.accounts
    where c.id in :ids
""")
List<Customer> findCustomersWithAccountsByIds(List<Long> ids);
```

Sau đó sort lại theo thứ tự ids nếu cần.

Đây là pattern rất thực tế trong dự án lớn:

Page parent IDs trước.
Fetch graph sau.



Nếu chỉ là API list, DTO projection còn gọn hơn.

13. LazyInitializationException là gì?

Lỗi này thường gặp:

```
failed to lazily initialize a collection of role:
could not initialize proxy – no Session
```



Ví dụ:

```
public Account getAccount(Long id) {  
    return accountRepository.findById(id).orElseThrow();  
}
```

Controller:

```
@GetMapping("/accounts/{id}")  
public AccountResponse get(@PathVariable Long id) {  
    Account account = accountService.getAccount(id);  
    return new AccountResponse(  
        account.getId(),  
        account.getCustomer().getName()  
    );  
}
```

Nếu transaction/persistence context đã đóng sau khi service return, lúc controller gọi:

```
account.getCustomer().getName()
```

Hibernate muốn lazy load customer nhưng session đã đóng. Thế là lỗi.

Một số app bật Open Session In View nên lỗi này không xuất hiện, vì session còn mở đến tận view/controller serialization. Spring Boot docs mô tả Open EntityManager in View cho web app là để cho phép lazy loading trong web views và có thể tắt bằng `spring.jpa.open-in-view=false`.  Home

Nhưng thực tế enterprise thường không muốn dựa vào OSIV để che lỗi fetch plan. Vì nếu controller hoặc Jackson serialization tự động đụng vào lazy field, query có thể bắn ra rất muộn, khó kiểm soát, và có thể gây N+1 ngoài service layer.

Cách đúng hơn:

Trong service method, xác định response cần gì.
Fetch đúng dữ liệu đó.
Map sang DTO trong transaction.
Trả DTO ra ngoài.



Ví dụ:

```
@Transactional(readOnly = true)  
public AccountResponse getAccount(Long id) {  
    Account account = accountRepository.findByIdWithCustomer(id)
```



```

        .orElseThrow();

    return new AccountResponse(
        account.getId(),
        account.getAccountNumber(),
        account.getCustomer().getName()
    );
}

```

Repository:

```

@Query("""
    select a
    from Account a
    join fetch a.customer
    where a.id = :id
""")
Optional<Account> findByIdWithCustomer(Long id);

```

14. Sai lầm rất hay gặp: trả thẳng Entity ra API

Ví dụ:

```

@GetMapping("/customers")
public List<Customer> getCustomers() {
    return customerRepository.findAll();
}

```

Cái này nguy hiểm.

Vì Jackson serialize object thành JSON, nó có thể gọi getter:

```

getAccounts()
getCards()
getAddress()
getTransactions()

```

Mỗi getter có thể kích hoạt lazy loading.

Kết quả:

API nhìn đơn giản nhưng query bắn lung tung.
Có thể N+1.
Có thể `LazyInitializationException`.
Có thể vòng lặp JSON vô hạn nếu `bidirectional relationship`.
Có thể lộ data nhạy cảm.



Trong production, nên trả DTO:

```
public record CustomerResponse(  
    Long id,  
    String name,  
    String email  
) {}
```

Service map rõ:

```
return customers.stream()  
    .map(c -> new CustomerResponse(c.getId(), c.getName(), c.getEmail()))  
    .toList();
```

Hoặc query projection trực tiếp.

15. Khi nào dùng Lazy, khi nào dùng Eager?

Thực tế mình khuyên:

Entity mapping: gần như luôn để LAZY.
Query/use case: cần gì fetch đó.



Không nên set EAGER vì “chỗ nào cũng cần”. Hôm nay API A cần customer, ngày mai API B chỉ cần account number. Nếu mapping là EAGER, API B vẫn bị kéo customer.

Tư duy đúng:

```
@ManyToOne(fetch = FetchType.LAZY)  
private Customer customer;
```

Sau đó mỗi use case tự quyết:

Use case cần account thôi:

```
List<Account> findByStatus(AccountStatus status);
```

Use case cần account + customer:

```
@Query("""
    select a
    from Account a
    join fetch a.customer
    where a.status = :status
""")
List<Account> findByStatusWithCustomer(AccountStatus status);
```

Use case chỉ cần summary:

```
@Query("""
    select new com.example.AccountSummaryDto(
        a.id, a.accountNumber, c.name
    )
    from Account a
    join a.customer c
    where a.status = :status
""")
List<AccountSummaryDto> findSummaries(AccountStatus status);
```

Đó là cách senior thiết kế: **fetch plan** nằm theo use case, không nằm cứng trong entity mapping.

16. Một ví dụ hoàn chỉnh: sửa N+1 trong service thật

Code bị lỗi N+1

```
@Service
@RequiredArgsConstructor
public class AccountQueryService {

    private final AccountRepository accountRepository;

    @Transactional(readOnly = true)
    public List<AccountSummaryResponse> getActiveAccounts() {
```

```

        List<Account> accounts = accountRepository.findByStatus(AccountStatus.ACTIVE);

        return accounts.stream()
            .map(account -> new AccountSummaryResponse(
                account.getId(),
                account.getAccountNumber(),
                account.getCustomer().getFullName()
            ))
            .toList();
    }
}

```

Repository:

```

public interface AccountRepository extends JpaRepository<Account, Long> {
    List<Account> findByStatus(AccountStatus status);
}

```

SQL thực tế:

```

select *
from account
where status = 'ACTIVE';

```

Sau đó N query:

```

select *
from customer
where id = ?;

```

Cách sửa 1: JOIN FETCH

```

public interface AccountRepository extends JpaRepository<Account, Long> {

    @Query("""
        select a
        from Account a
        join fetch a.customer
        where a.status = :status
        """)
}

```

```
List<Account> findByStatusWithCustomer(AccountStatus status);
}
```

Service:

```
@Transactional(readOnly = true)
public List<AccountSummaryResponse> getActiveAccounts() {
    List<Account> accounts = accountRepository.findByStatusWithCustomer(AccountStatus.ACTIVE);

    return accounts.stream()
        .map(account -> new AccountSummaryResponse(
            account.getId(),
            account.getAccountNumber(),
            account.getCustomer().getFullName()
        ))
        .toList();
}
```

Tốt khi bạn vẫn muốn dùng entity.

Cách sửa 2: DTO projection

```
public interface AccountRepository extends JpaRepository<Account, Long> {

    @Query("""
        select new com.example.AccountSummaryResponse(
            a.id,
            a.accountNumber,
            c.fullName
        )
        from Account a
        join a.customer c
        where a.status = :status
    """)
    List<AccountSummaryResponse> findAccountSummaries(AccountStatus status);
}
```

Service:

```
@Transactional(readOnly = true)
public List<AccountSummaryResponse> getActiveAccounts() {
    return accountRepository.findAccountSummaries(AccountStatus.ACTIVE);
}
```

Cách này thường tốt nhất cho API list vì không load entity thừa.

17. Với OneToMany thì xử lý thế nào?

Giả sử:

```
@Entity
public class Customer {

    @Id
    private Long id;

    private String name;

    @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
    private List<Account> accounts;
}
```

API cần trả:

```
{
  "customerId": 1,
  "name": "Nguyen Van A",
  "accounts": [
    {"id": 10, "accountNumber": "123"},
    {"id": 11, "accountNumber": "456"}
  ]
}
```

Nếu bạn làm:

```
List<Customer> customers = customerRepository.findAll();

for (Customer customer : customers) {
    customer.getAccounts().size();
}
```

Lại N+1:

1 query customer
+ N query accounts



Có thể dùng:

```
@Query("""
    select distinct c
    from Customer c
    left join fetch c.accounts
    where c.status = :status
""")
List<Customer> findByStatusWithAccounts(CustomerStatus status);
```

Nhưng nếu có pagination, đừng join fetch collection trực tiếp với Pageable nếu không hiểu rõ hậu quả. Dùng two-step:

```
@Query("""
    select c.id
    from Customer c
    where c.status = :status
    order by c.createdAt desc
""")
Page<Long> findIdsByStatus(CustomerStatus status, Pageable pageable);
```

Rồi:

```
@Query("""
    select distinct c
    from Customer c
    left join fetch c.accounts
    where c.id in :ids
""")
List<Customer> findWithAccountsByIdIn(List<Long> ids);
```

Hoặc query DTO dạng flat rồi group bằng Java:

```
customer_id | customer_name | account_id | account_number
```

Sau đó map thành nested response.

Với API public, cách DTO flat + group thường rất kiểm soát được performance.

18. Cách phát hiện N+1 trong dự án thật

Đừng đoán. Hãy bật SQL log hoặc dùng tool.

Trong dev:

```
spring.jpa.show-sql=true  
spring.jpa.properties.hibernate.format_sql=true
```

Nhưng production không nên bật full SQL log bữa vì log rất nặng và có thể lộ data.

Tốt hơn dùng:

```
p6spy  
datasource-proxy  
Hibernate statistics  
APM như Datadog/New Relic  
slow query log của DB  
integration test assert số query
```



Hibernate docs cũng gợi ý dùng DataSource proxy để assert số statement ở test time, giúp integration test fail khi N+1 query tự động xuất hiện.  [Hibernate Docum...](#)

Ví dụ test mindset:

```
API /accounts?page=0&size=20 đáng lẽ chạy 1-2 queries.  
Nếu sau khi sửa code nó chạy 42 queries, test phải fail.
```



Trong team lớn, đây là cách bắt lỗi sớm trước khi lên production.

19. Một lỗi rất tinh vi: toString(), equals(), hashCode() đùng vào lazy field

Ví dụ dùng Lombok:

```
@Data  
@Entity  
public class Customer {  
  
    @Id  
    private Long id;
```



```
private String name;

@OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
private List<Account> accounts;
}
```

@Data sinh toString, equals, hashCode có thể dùng cả accounts.

Khi log:

```
log.info("customer={}", customer);
```

Nó gọi toString(), độn vào accounts, Hibernate lazy load collection. Thế là tự nhiên log một object lại bắn query.

Best practice:

```
@Getter
@Setter
@Entity
public class Customer {

    @Id
    private Long id;

    private String name;

    @ToString.Exclude
    @EqualsAndHashCode.Exclude
    @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
    private List<Account> accounts;
}
```

Hoặc tự viết equals/hashCode chỉ dựa trên id/business key phù hợp.

20. Một lỗi khác: mapping sai chiều quan hệ

Ví dụ @OneToOne hai chiều lazy có thể phức tạp hơn ManyToOne. Hibernate docs nói với bidirectional @OneToOne, để biết có associated record phía child hay không, Hibernate có thể phải fetch child bằng secondary query, và điều đó có thể dẫn tới N+1; tài liệu gợi ý dùng unidirectional @OneToOne với @MapsId hiệu quả hơn trong case đó.

Thực tế: nếu không cần bidirectional relationship, đừng map hai chiều chỉ vì "cho tiện".

Ví dụ nhiều dự án map kiểu:

```
Customer -> List<Account>
Account -> Customer
```

Nhưng API chỉ cần từ account sang customer, không bao giờ từ customer sang accounts. Vậy không nhất thiết phải map List<Account> trong Customer.

Mapping càng nhiều chiều, object graph càng dễ bị lazy load dây chuyền.

21. Best practice thực tế cho senior Java

Thứ nhất: Entity mapping mặc định LAZY

```
@ManyToOne(fetch = FetchType.LAZY)
@OneToOne(fetch = FetchType.LAZY)
@OneToMany(fetch = FetchType.LAZY)
@ManyToMany(fetch = FetchType.LAZY)
```

Riêng @ManyToOne và @OneToOne trong JPA nếu không set có thể dễ bị eager theo default, nên trong team nên có convention là **ghi rõ** fetch = FetchType.LAZY .

Thứ hai: Không trả entity trực tiếp ra controller

Controller nên trả DTO.

Entity là model persistence.
DTO là contract API.



Không trộn hai cái này.

Thứ ba: Mỗi API phải có fetch plan riêng

API detail có thể dùng JOIN FETCH hoặc EntityGraph .

API list/search thường dùng DTO projection.

Batch/report lớn nên dùng projection hoặc native query nếu cần.

Thứ tư: Không dùng EAGER để "chữa cháy"

EAGER làm query khó kiểm soát hơn. Nó có thể load data không cần, gây N+1 trong JPQL, và không override được per query theo cách linh hoạt. Hibernate docs cũng khuyến nghị tránh eager và đánh dấu association lazy by default.  [Hibernate Docum...](#)

Thứ năm: Với collection lớn, cần thận JOIN FETCH

Nếu fetch OneToMany, nghĩ ngay đến:

Có pagination không?
Collection có lớn không?
Có fetch nhiều collection cùng lúc không?
Có bị duplicate parent rows không?
Có cần two-step query không?



Thứ sáu: Test số query

Một senior không chỉ nhìn code. Senior nhìn SQL.

Một method service quan trọng nên biết:

Nó bắn mấy query?
Query có dùng index không?
Có query trong vòng lặp không?
Có query phát sinh khi serialize JSON không?



22. Câu trả lời phỏng vấn senior gọn nhưng mạnh

Bạn có thể trả lời như sau:

Lazy và eager trong JPA/Hibernate không chỉ là load chậm hay load sớm, mà là fetch 
Em không dùng EAGER như cách chữa N+1, vì eager làm association luôn bị load kể
Nếu API cần entity và relation nhỏ như ManyToOne thì em dùng JOIN FETCH hoặc EAGER

23. Tóm lại bằng một câu dễ nhớ

Lazy không xấu.
Eager không phải giải pháp.
N+1 xảy ra khi fetch plan không khớp với dữ liệu mà use case thật sự cần.
Senior không chọn lazy/eager theo cảm tính; senior thiết kế query theo response



    ...  Nguồn